PLANNEX

The complete code learning manual

A visual, beginner-friendly walkthrough of every HTML, CSS, and JavaScript file in the student planner.

LEARN THE STRUCTURE / FOLLOW REAL CODE / EDIT WITH CONFIDENCE

Midnight	Surface	Green	Cloud	Slate
#17182F	#242642	#8DDA7A	#F5F7FA	#667085

This guide matches the current Plannex source files. Each lesson explains what the section does, how it connects to the other files, and what to edit when you want to change it. The final appendices reproduce the full source files with line numbers.

Start with the edit map on the next page. When you are ready to learn, read the chapters in order and compare every example with the complete file listing at the back.

How to use this manual

You do not need to memorize the whole project. Learn the job of each file first, then follow a feature from its HTML control to its JavaScript behavior and CSS appearance.

File	What it controls	Open it when you want to change
index.html	The page structure, headings, buttons, input areas, and named sections.	Words, page content, labels, button locations, or accessible names.
styles.css	Colors, spacing, fonts, cards, grids, dialogs, and responsive layout.	Brand colors, sizes, visual style, tablet layout, or phone layout.
app.js	Sample data, rendering, calculations, events, dialogs, and browser saving.	Tasks, classes, expenses, grade behavior, interactions, or saved data.
CODE_GUIDE.md	A quick edit map maintained next to the app.	A short reminder of where a common change belongs.

Contents

Section	Pages
Part 1 - Understand the three files	3-4
Part 2 - HTML screens and controls	5-9
Part 3 - CSS appearance and responsive layout	10-24
Part 4 - JavaScript data, rendering, and behavior	25-33
Part 5 - Practice edits, debugging, glossary	34-35
Appendix A - Complete index.html	36-38
Appendix B - Complete styles.css	39-55
Appendix C - Complete app.js	56-62

Reading code in this PDF: line numbers refer to the source file as it was included here. Long lines may wrap visually. The PDF uses plain hyphens in place of typographic dash characters in code samples; your editable files are unchanged.

PART 1 / Understand the three files

Think of the app as three layers working together. HTML names the things on the page. CSS makes those things look right. JavaScript fills them with information and responds when you click, type, or save.

Layer	Simple analogy	Plannex example
HTML	The rooms and furniture in a house.	A button that says Add a task and a section with id page-planner.
CSS	The paint, spacing, and arrangement.	The green action button, navy sidebar, or a two-column page.
JavaScript	The instructions for how the house works.	Opening a dialog, adding a task, recalculating the grade, saving changes.

Follow one feature through the app

When you add a task, the button in index.html carries data-action="add-task". app.js listens for clicks and passes that action to openDialog(). The dialog asks for a title, due text, and priority. On Save, JavaScript creates a task record, rerenders the page, and stores the updated state. CSS styles the button, dialog, fields, and task row.

HTML: the action the button requests

```
70                <p class="page-subtitle">A calm place to keep your student life together.</p>
71              </div>
72              <button class="button button-dark" data-action="add-task"><span> </span> Add a task</button>
73            </div>
74
```

JavaScript: one click listener finds the action button

```
457   // Delegate clicks because task, class, and grade controls are rebuilt during rendering.
458   document.addEventListener("click", (e) => {
459     const target = e.target.closest("[data-action]");
460     if (target) openDialog(target.dataset.action);
461     const toggle = e.target.closest("[data-toggle-task]");
462     if (toggle) {
463       const task = state.tasks.find((t) => t.id === toggle.dataset.toggleTask);
464       if (task) task.done = !task.done;
```

JavaScript: the shared dialog chooses fields for add-task

```
557   function openDialog(action) {
558     const config = {
559       "add-task": {
560         title: "Add a task",
561         subtitle: "A SMALL THING, DONE WELL",
562         fields: `${field("What needs doing?", "title", "text", "", "", 'placeholder="e.g. Review lecture notes"')}<div class="dialog-grid">${fiel
      d("Due", "due", "text", "Today · Anytime", "", 'placeholder="Today · 3:00 PM"')}${field("Priority", "priority", "text", "Medium", ["Low", "Medium
      ", "High"])}</div>`,
563         save: (data) =>
564           state.tasks.unshift({
565             id: makeId(),
566             title: data.get("title").trim(),
567             due: data.get("due").trim() || "Today",
568             priority: data.get("priority"),
569             done: false,
570           }),
571       },
572       "add-expense": {
573         title: "Add an expense",
574         subtitle: "MONEY CHECK-IN",
575         fields: `${field("What did you spend on?", "name", "text", "", "", 'placeholder="e.g. Lunch"')}<div class="dialog-grid">${field("Amount (
      ₱)", "amount", "number", "", "", 'min="1"')}${field("Category", "category", "text", "Food", ["Food", "Transport", "School", "Personal", "Other"])
      }</div>${field("Date", "date", "date", today.toISOString().slice(0, 10))}`,
576         save: (data) =>
577           state.expenses.unshift({
578             id: makeId(),
579             name: data.get("name").trim(),
580             amount: Number(data.get("amount")),
581             category: data.get("category"),
582             date: data.get("date"),
583           }),
584       },
585       "add-class": {
```

The main data path

Sample data or saved data becomes the JavaScript state. The render() function updates the HTML elements from that state. User actions change the state, and render() runs again. persist() saves the state in browser local storage so a refresh can restore it.

Step	What happens	Where to look
1. Start	Load saved state, or copy the sample seed.	app.js, sections 1 and 2
2. Show	Write totals and rows into named HTML elements.	app.js, section 3
3. Interact	Listen for clicks, changes, and form submission.	app.js, sections 4 and 5
4. Save	Serialize state into localStorage.	app.js, persist()

If a control looks right but does nothing, trace its data-action or id into app.js. If it works but looks wrong, find its class in styles.css. If the wording is wrong, start in index.html.

PART 2 / HTML: the screens and controls

The HTML file is the visible skeleton. The comments divide it into areas. The id values provide stable names for JavaScript, and the class values let CSS style groups of similar elements.

Lesson 1 - Document setup and shared frame

The opening lines tell the browser this is modern HTML, set the text language and phone viewport, provide tab metadata, and connect the font service and stylesheet. The sidebar and top bar are shared by all pages, so they sit outside the individual page sections.

Open index.html, lines 1-63

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="theme-color" content="#17182F">
    <meta name="description" content="Plannex is your calm, everyday student planner for classes, tasks, spending, and grades.">
    <title>Plannex — your student planner</title>

    <!-- Fonts and the main visual style sheet. -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Manrope:wght@400;500;600;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <div class="app-shell">
      <!-- Shared sidebar: the data-page names must match the page section IDs below. -->
      <aside class="sidebar" id="sidebar">
        <a class="brand" href="#overview" aria-label="Plannex home">
          <span class="brand-mark"><svg viewBox="300 110 420 420" aria-hidden="true"><image href="assets/plannex-logo.png" width="1110" height="835"></image></svg></span><span class="brand-name">PLANN<span class="brand-green">E</span>X<small>Make every day Count</small></span>
        </a>

        <p class="nav-label">Workspace</p>
        <nav class="primary-nav" aria-label="Main navigation">
          <a class="nav-link active" href="#overview" data-page="overview"><span class="nav-icon"> </span>Overview</a>
          <a class="nav-link" href="#planner" data-page="planner"><span class="nav-icon"> </span>Daily planner</a>
          <a class="nav-link" href="#classes" data-page="classes"><span class="nav-icon"> </span>Class schedule</a>
          <a class="nav-link" href="#budget" data-page="budget"><span class="nav-icon"> </span>My budget</a>
          <a class="nav-link" href="#grades" data-page="grades"><span class="nav-icon"> </span>Grade calculator</a>
        </nav>

        <div class="sidebar-bottom">
          <div class="sidebar-note">
            <span class="note-spark"> </span>
            <p>One thing at a time.<br><strong>You've got this.</strong></p>
            <span class="note-doodle"> </span>
          </div>
          <div class="profile">
            <div class="avatar">JS</div>
            <div><strong>Jamie Santos</strong><small>Student workspace</small></div>
            <span class="profile-dots">···</span>
          </div>
        </div>
      </aside>

      <main class="main-area">
        <!-- Shared header: JavaScript updates the page name and today's date. -->
        <header class="topbar">
          <button class="mobile-menu" id="menu-toggle" aria-label="Open menu"> </button>
          <div class="breadcrumb">
            <span>My workspace</span>
            <b>/</b>
            <strong id="crumb-current">Overview</strong>
          </div>
          <div class="topbar-right">
            <span class="today-chip"><span class="live-dot"></span><span id="today-label"></span></span>
            <button class="icon-button" id="reset-data" title="Restore sample data" aria-label="Restore sample data"> </button>
            <div class="avatar avatar-small">JS</div>
          </div>
        </header>

        <div class="content-wrap">
```

- The doctype and html element start the document. lang="en" helps assistive technology pronounce it.
- The viewport meta tag tells a phone to use its real device width instead of shrinking a desktop layout.
- A class such as sidebar can be reused for styling. An id such as sidebar identifies one unique element for JavaScript.
- Navigation href values such as #planner are familiar browser links. data-page="planner" gives app.js the same page name.

- The shared header contains the current page label and today's date. JavaScript updates these values.

How navigation names connect

Menu link data-page	Page section id	Visible label
overview	page-overview	Overview
planner	page-planner	Daily planner
classes	page-classes	Class schedule
budget	page-budget	My budget
grades	page-grades	Grade calculator

If you rename a page, change the menu data-page, its matching page-section id, and the page label map in navigate(). Otherwise the app may show the wrong section or fall back to Overview.

Lesson 2 - Overview dashboard

The Overview is a summary, not a separate data store. Its number slots and cards have ids that JavaScript updates from the same state used by the other pages. For example, stat-tasks, stat-budget, and stat-grade are targets for summary values. The today task list is filled into overview-tasks.

Open the Overview section

```
64          <!-- Overview: a quick summary of tasks, classes, budget, and study momentum. -->
65          <section class="page-section active-page" id="page-overview">
66            <div class="welcome-row">
67              <div>
68                <p class="eyebrow" id="greeting">YOUR DAY, AT A GLANCE</p>
69                <h1>A little more <span>organized.</span></h1>
70                <p class="page-subtitle">A calm place to keep your student life together.</p>
71              </div>
72              <button class="button button-dark" data-action="add-task"><span> </span> Add a task</button>
73            </div>
74
75            <div class="stats-grid">
76              <article class="stat-card">
77                <div class="stat-top"><span>Tasks for today</span><span class="stat-icon icon-green"> </span></div>
78                <div class="stat-number" id="stat-tasks">0 <small>tasks left</small></div>
79                <div class="progress-track"><span id="task-progress"></span></div>
80                <div class="stat-foot" id="task-foot">Keep your momentum going</div>
81              </article>
82              <article class="stat-card">
83                <div class="stat-top"><span>Budget remaining</span><span class="stat-icon icon-peach"> </span></div>
84                <div class="stat-number" id="stat-budget">₱0</div>
85                <div class="stat-foot"><span class="mini-dot"></span><span id="budget-foot">of your monthly allowance</span></div>
86              </article>
87              <article class="stat-card">
88                <div class="stat-top"><span>Current raw grade</span><span class="stat-icon icon-lilac"> </span></div>
89                <div class="stat-number" id="stat-grade">— <small>%</small></div>
90                <div class="stat-foot">From your entered score totals</div>
91              </article>
92            </div>
93
94            <div class="dashboard-grid">
95              <article class="panel today-panel">
96                <div class="panel-heading">
97                  <div><p class="eyebrow">MAKE IT COUNT</p><h2>Today's plan <span class="heading-count" id="today-count">0</span></h2></div>
98                  <a href="#planner" class="text-link">See all <span>→</span></a>
99                </div>
100               <div class="task-list" id="overview-tasks"></div>
101               <button class="add-inline" data-action="add-task">  Add a task</button>
102             </article>
103
104             <article class="panel schedule-panel">
105               <div class="panel-heading">
106                 <div><p class="eyebrow">UP NEXT</p><h2>Your schedule</h2></div>
107                 <a href="#classes" class="text-link">Full week <span>→</span></a>
108               </div>
109               <div class="next-class" id="next-class"></div>
110             </article>
111
112             <article class="panel spending-panel">
113               <div class="panel-heading">
114                 <div><p class="eyebrow">MONEY CHECK-IN</p><h2>This month</h2></div>
115                 <a href="#budget" class="text-link">Budget <span>→</span></a>
116               </div>
117               <div class="spend-total"><strong id="overview-spent">₱0</strong><span>spent so far</span></div>
118               <div class="budget-bar"><span id="budget-bar"></span></div>
119               <div class="budget-labels"><span id="spent-label">₱0 spent</span><span id="limit-label">₱0 limit</span></div>
120               <div class="recent-transactions" id="recent-transactions"></div>
121             </article>
122
123             <article class="panel focus-panel">
```

```
                <div class="focus-copy">
                  <span class="focus-badge">A SMALL REMINDER</span>
                  <h2>Progress is still progress, <em>even on the busy days.</em></h2>
                  <p>Pick one thing to start. The rest can follow.</p>
                </div>
                <div class="focus-art"><span class="art-sun"></span><span class="art-star"> </span><span class="art-plant"> </span></div>
              </article>
            </div>
          </section>

```

- A section with id="page-overview" is the container for the first screen. The active-page class marks the section that is currently visible.
- The Add a task buttons use data-action="add-task". The repeated action name means they can share one JavaScript handler.
- Empty div elements such as next-class and recent-transactions are placeholders. JavaScript inserts the current content into them.
- The CSS class grid or card describes appearance. The id identifies a target or unique control.

Lesson 3 - Daily planner

The planner page contains a task list target, a count, and three filter buttons. The filter buttons share the class filter-tab and carry data-filter values: all, open, and done. JavaScript uses the selected value to choose which task rows to display.

Open the Daily planner section

```
          <!-- Daily planner: filter tasks by all, still to do, or completed. -->
          <section class="page-section" id="page-planner">
            <div class="welcome-row">
              <div>
                <p class="eyebrow">YOUR EVERYDAY RESET</p>
                <h1>Daily <span>planner.</span></h1>
                <p class="page-subtitle">A simple list is a good place to begin.</p>
              </div>
              <button class="button button-dark" data-action="add-task">  Add a task</button>
            </div>

            <div class="planner-layout">
              <article class="panel planner-main">
                <div class="panel-heading">
                  <div><p class="eyebrow" id="planner-date">TODAY</p><h2>My to-do list</h2></div>
                  <span class="heading-count" id="planner-count">0</span>
                </div>
                <div class="filter-tabs">
                  <button class="filter-tab selected" data-filter="all">All</button>
                  <button class="filter-tab" data-filter="open">To do</button>
                  <button class="filter-tab" data-filter="done">Completed</button>
                </div>
                <div class="task-list full-task-list" id="planner-tasks"></div>
              </article>

              <aside class="panel planner-side">
                <p class="eyebrow">A GOOD RHYTHM</p>
                <h2>Make space for breaks.</h2>
                <p class="page-subtitle">Try a short focus session, then step away for a few minutes. Small resets add up.</p>
                <div class="break-card"><span> </span><div><strong>Focus for 25 minutes</strong><small>One task. No rush.</small></div></div>
                <div class="week-mini" id="week-mini"></div>
              </aside>
            </div>
          </section>

```

The task rows themselves are generated by app.js, not written one by one in HTML. HTML only provides the empty containers where those rows will appear.

Lesson 4 - Class schedule

The timetable is assembled in JavaScript because its rows depend on the DAYS and HOURS arrays. HTML provides a scroll wrapper and an empty element with id timetable. The Overview also has a separate next-class placeholder.

Open the Class schedule section

```
          <!-- Class schedule: JavaScript builds the weekly grid from the class data. -->
          <section class="page-section" id="page-classes">
            <div class="welcome-row">
              <div>
                <p class="eyebrow">YOUR WEEK, IN ONE PLACE</p>
                <h1>Class <span>schedule.</span></h1>
```

```
                <p class="page-subtitle">Know where you need to be, and when.</p>
              </div>
              <button class="button button-dark" data-action="add-class">  Add a class</button>
            </div>

            <article class="panel timetable-panel">
              <div class="panel-heading">
                <div><p class="eyebrow">SEMESTER 1 · 2026</p><h2>Weekly timetable</h2></div>
                <div class="schedule-legend"><span><i class="legend-dot dot-green"></i>Classes</span></div>
              </div>
              <div class="table-scroll"><div class="timetable" id="timetable"></div></div>
            </article>
            <p class="section-hint">Tip: classes repeat each week. Add your own subjects to make this schedule yours.</p>
          </section>

```

Lesson 5 - Budget and grade screens

The budget screen has named targets for allowance, total spending, remaining money, filter selection, and expense rows. The grade screen has a table body for assessment rows and buttons to add a row or compute a result. JavaScript updates these areas; CSS provides the tables and cards.

Open the Budget section

```
          <!-- Budget: allowance and expenses are stored together in the planner state. -->
          <section class="page-section" id="page-budget">
            <div class="welcome-row">
              <div>
                <p class="eyebrow">SPEND WITH A LITTLE MORE CLARITY</p>
                <h1>My <span>budget.</span></h1>
                <p class="page-subtitle">Keep track of what comes in and where it goes.</p>
              </div>
              <button class="button button-dark" data-action="add-expense">  Add expense</button>
            </div>

            <div class="budget-stats">
              <article class="stat-card budget-stat">
                <span>Monthly allowance</span>
                <strong id="budget-total">₱0</strong>
                <button class="subtle-button" data-action="edit-budget">Edit allowance →</button>
              </article>
              <article class="stat-card budget-stat"><span>Total spent</span><strong id="budget-spent">₱0</strong><small>This month</small></arti
cle>
              <article class="stat-card budget-stat"><span>Left to spend</span><strong id="budget-remaining">₱0</strong><small id="budget-remaini
ng-note">You’re doing great</small></article>
            </div>

            <article class="panel expenses-panel">
              <div class="panel-heading">
                <div><p class="eyebrow">THE LITTLE THINGS ADD UP</p><h2>Recent expenses</h2></div>
                <select id="expense-filter" class="select-small" aria-label="Filter expenses">
                  <option value="all">All categories</option>
                  <option>Food</option>
                  <option>Transport</option>
                  <option>School</option>
                  <option>Personal</option>
                  <option>Other</option>
                </select>
              </div>
              <div id="expense-list" class="expense-list"></div>
            </article>
          </section>

```

Open the Grade calculator section

```
          <!-- Grade calculator: enter assessment points, then compute the raw grade. -->
          <section class="page-section" id="page-grades">
            <div class="welcome-row">
              <div>
                <p class="eyebrow">KEEP AN EYE ON YOUR PROGRESS</p>
                <h1>Grade <span>calculator.</span></h1>
                <p class="page-subtitle">Enter the score and total points for each assessment.</p>
              </div>
            </div>

            <div class="grade-summary">
              <div>
                <span class="eyebrow">OVERALL RAW GRADE</span>
                <div class="grade-big" id="grade-average">–<small>%</small></div>
                <p class="grade-caption" id="grade-caption">Enter scores and totals, then compute your grade.</p>
              </div>
              <div class="grade-summary-art"><span> </span></div>
            </div>

            <article class="panel grades-panel">
              <div class="panel-heading">
                <div><p class="eyebrow">ASSESSMENT ITEMS</p><h2>Grade tracker</h2></div>
                <span class="grade-count" id="grade-count">0 rows</span>
              </div>
              <div class="grades-table-wrap">
                <table class="grades-table">
```

```
                  <thead>
                    <tr><th>CATEGORY</th><th>SCORE</th><th>TOTAL</th><th>RAW GRADE</th><th></th></tr>
                  </thead>
                  <tbody id="grade-rows"></tbody>
                </table>
              </div>
              <div class="grade-actions">
                <button class="button button-quiet" id="add-grade-row" type="button">  Add row</button>
                <button class="button button-dark" id="compute-grade" type="button">Compute grade</button>
              </div>
              <p class="grade-note">Raw grade = points earned ÷ points possible × 100. All entered items contribute by their point totals.</p>
            </article>
          </section>

```

- An input/select id connects user entry to a specific app.js event listener.
- A button id is best for a unique action. data-action is used for repeated actions that can appear in multiple places.
- The grade table is table structure. Its tbody stays empty until renderGrades() inserts rows.

Lesson 6 - Shared dialog, cancel, and feedback

The dialog is outside the five page sections, so it can open over any page. It contains a title, a placeholder for fields, a Cancel button, and a Save button. The close buttons use data-close-dialog; the form uses a submit handler for Save. This separation is why Cancel dismisses the form without saving it.

The shared dialog and script loading

```
          <footer class="footer-note">Made for the little things that make student life feel easier. <span> </span></footer>
        </div>
      </main>
    </div>

    <!-- Shared feedback message and entry dialog used by task, class, and budget actions. -->
    <div class="toast" id="toast" role="status" aria-live="polite"></div>
    <dialog id="entry-dialog">
      <form id="entry-form">
        <div class="dialog-head">
          <div><p class="eyebrow" id="dialog-eyebrow">QUICK ADD</p><h2 id="dialog-title">Add a task</h2></div>
          <button class="close-dialog" type="button" data-close-dialog aria-label="Close">×</button>
        </div>
        <div id="dialog-fields"></div>
        <div class="dialog-actions">
          <button class="button button-quiet" type="button" data-close-dialog>Cancel</button>
          <button class="button button-dark" id="dialog-save" type="submit">Save</button>
        </div>
      </form>
    </dialog>

    <!-- Load behavior after all page controls are in place. -->
    <script src="app.js"></script>
  </body>
</html>
```

- The dialog element provides a native modal panel. showModal() opens it above the current page.
- The Cancel button has type="button" so it cannot submit the form by accident.
- The Save button is type="submit". JavaScript prevents the browser's default page reload, validates, saves the record, and closes the dialog.
- role="status" and aria-live="polite" let assistive technology announce toast feedback without interrupting the user.

When adding a new visible control, include a text label or accessible name. If a control is inserted by JavaScript, give it the same care as static HTML.

PART 3 / CSS: the appearance and responsive layout

CSS rules follow a pattern: a selector chooses elements, declarations choose properties, and values determine the appearance. A period targets a class, a hash targets an id, and a colon can target a state such as hover or focus.

CSS lines	Style group and purpose
1-63	Palette tokens and shared browser defaults.
64-415	Logo, sidebar, navigation, top bar, page frame, buttons, and page visibility.
416-564	Overview metric cards, progress bars, and panel headings.
565-914	Task rows, next-class preview, spending preview, focus card artwork, and footer.
915-1391	Planner, timetable, full budget list, grade summary, and grade table.
1392-1496	Shared entry dialog, fields, backdrop, and toast.
1497-1755	Tablet and phone breakpoints plus reduced-motion support.

Lesson 7 - Palette, tokens, and browser defaults

The :root block defines design tokens. A token is a named value that can be reused. Change a palette value once and every rule using that variable updates. The early rules also set the font, box sizing, body background, and shared heading defaults.

The foundation at the top of styles.css

```
/* Brand palette and shared design tokens */

:root {
  --midnight: #17182F;
  --surface: #242642;
  --green: #8DDA7A;
  --cloud: #F5F7FA;
  --slate: #667085;
  --ink: #24263d;
  --muted: #898da0;
  --line: #eceef3;
  --white: #fff;
  --peach: #f8e3d1;
  --lilac: #e8e1fa;
  --mint: #e5f5df;
  --shadow: 0 8px 28px rgba(23,24,47,.045);
}

* {
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  background: var(--cloud);
  color: var(--ink);
  font-family: 'DM Sans',sans-serif;
  font-size: 14px;
  -webkit-font-smoothing: antialiased;
}

button,input,select {
  font: inherit;
}

button,a {
  -webkit-tap-highlight-color: transparent;
}

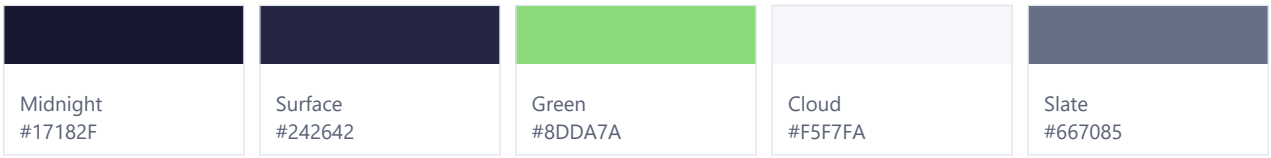
button {
  cursor: pointer;
}

.app-shell {
  min-height: 100vh;
}

.sidebar {
```

```
  width: 244px;
  position: fixed;
  inset: 0 auto 0 0;
  background: var(--midnight);
  color: #fff;
  padding: 28px 18px 18px;
  display: flex;
  flex-direction: column;
  z-index: 20;
}

```



- var(--midnight) reads the value stored in --midnight.
- The universal box-sizing rule makes declared widths include padding and borders, which simplifies layout math.
- A CSS selector such as body sets a default for the whole document. A more specific selector can override it later.
- Keep the five brand colors here so visual changes stay consistent across all sections.

Lesson 8 - Brand, sidebar, navigation, and page frame

The next CSS group styles the logo lockup and fixed sidebar, then the navigation links, mobile menu, top bar, page title, and section visibility. The page-section class defines a page; .active-page changes its display so only the selected page is shown.

Beginning of the brand and navigation rules

```
/* Brand lockup and fixed navigation shell */

.brand {
  display: flex;
  align-items: center;
  gap: 11px;
  color: #fff;
  text-decoration: none;
  font: 800 22px Manrope,sans-serif;
  letter-spacing: -1px;
  padding: 0 12px 38px;
}

.brand-mark {
  width: 40px;
  height: 40px;
  flex: none;
  overflow: hidden;
  border: 1px solid #343550;
  border-radius: 12px;
  background: var(--surface);
  box-shadow: 0 3px 11px #090a1b55;
}

.brand-mark svg {
  display: block;
  width: 100%;
  height: 100%;
}

.brand-name {
  font: 800 15px/1.1 Manrope,sans-serif;
  letter-spacing: 1.4px;
  color: #fff;
}

.brand-green {
  color: var(--green);
}

.brand-name small {
  display: block;
  margin-top: 5px;
  font: 400 9px/1 'DM Sans',sans-serif;
  letter-spacing: .15px;
  color: #a4a7b7;
}

.nav-label,.eyebrow {
  font-size: 10px;
  letter-spacing: 1.5px;
  font-weight: 700;
  color: #a4a7b7;
  margin: 0;
}
```

```
.nav-label {
  padding: 0 14px 12px;
  color: #797d94;
}

.primary-nav {
  display: grid;
  gap: 5px;
}

.nav-link {
  height: 43px;
  display: flex;
  align-items: center;
  gap: 13px;
  border-radius: 9px;
  color: #b9bbca;
  text-decoration: none;
  padding: 0 13px;
  font-size: 13px;
  font-weight: 500;
  transition: .2s;
}

.nav-link:hover {
  background: #22233d;
```

The page container and active-page behavior

```
  background: #f2f3f7;
  color: #55586c;
}

.page-section {
  display: none;
  animation: appear .25s ease;
}

.page-section.active-page {
  display: block;
}

@keyframes appear {
  from {
    opacity: .4;
    transform: translateY(4px);
  }

  to {
    opacity: 1;
    transform: none;
  }

}
```

- A fixed sidebar stays in one place while the main page scrolls.
- The active nav class marks the current menu item. JavaScript toggles it when navigating.
- Use a small scoped rule such as .sidebar .nav-link when a style should only affect links inside the sidebar.
- The logo image path in HTML must match the actual asset path. CSS can control its displayed width and height without changing the image file.

Lesson 9 - Dashboard cards and reusable task rows

The dashboard groups metric cards, progress bars, task rows, class cards, and spend summaries. Shared class names keep a task looking consistent whether it appears on the Overview or full planner page. State classes like task-done and checked change appearance when a task is completed.

Dashboard metrics and summary card styles

```
/* Dashboard summary cards */

.stats-grid {
  display: grid;
  grid-template-columns: repeat(3,1fr);
  gap: 15px;
  margin-bottom: 18px;
}

.stat-card,.panel {
  background: var(--white);
  border: 1px solid #eceef3;
  border-radius: 12px;
  box-shadow: var(--shadow);
}
```

```
.stat-card {
  padding: 17px 19px;
}

.stat-top {
  display: flex;
  justify-content: space-between;
  align-items: center;
  color: #797c8c;
  font-size: 11px;
}

.stat-icon {
  width: 29px;
  height: 29px;
  display: grid;
  place-items: center;
  border-radius: 8px;
  font-size: 15px;
}

.icon-green {
  background: #eaf7e5;
  color: #73b765;
}

.icon-peach {
  background: #fff0e5;
  color: #d68f63;
}

.icon-lilac {
  background: #f0eafb;
  color: #987cc8;
}

.stat-number {
  font: 700 25px Manrope,sans-serif;
  letter-spacing: -.8px;
  color: var(--midnight);
  margin: 8px 0 7px;
}

.stat-number small {
  font: 500 11px 'DM Sans',sans-serif;
  letter-spacing: 0;
  color: #9396a3;
}

.progress-track {
  height: 4px;
  background: #edf0ea;
  border-radius: 4px;
  overflow: hidden;
  margin: 11px 0 8px;
}

.progress-track span,.budget-bar span {
  height: 100%;
  display: block;
  background: var(--green);
  width: 0;
  transition: width .3s;
  border-radius: 4px;
}

.stat-foot {
  font-size: 10px;
  color: #a0a2ad;
  margin-top: 8px;
  display: flex;
  align-items: center;
  gap: 6px;
}

.dashboard-grid {
  display: grid;
  grid-template-columns: 1.1fr .9fr;
  gap: 16px;
}

.panel {
  padding: 19px;
}

.panel-heading {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 17px;
}

.panel-heading h2,.planner-side h2 {
  font: 700 16px Manrope,sans-serif;
  letter-spacing: -.35px;
  color: var(--midnight);
  margin: 6px 0 0;
}

.panel-heading .eyebrow {
  font-size: 9px;
```

```
}

.heading-count {
  display: inline-grid;
  place-items: center;
  background: #f0f1f5;
  border-radius: 20px;
  min-width: 21px;
  height: 21px;
  padding: 0 7px;
  font-size: 10px;
  color: #787b89;
  margin-left: 4px;
  vertical-align: 2px;
}

.text-link {
  font-size: 10px;
  color: #878a99;
  text-decoration: none;
}

.text-link span {
  color: #7ebc6e;
  margin-left: 4px;
  font-size: 14px;
}

.text-link:hover {
  color: var(--midnight);
}

```

Task list and task row styling

```
/* Reusable task rows and schedule cards */

.task-list {
  display: grid;
}

.task-row {
  display: flex;
  align-items: center;
  gap: 11px;
  min-height: 49px;
  border-bottom: 1px solid #f0f1f5;
  padding: 7px 0;
}

.task-row:last-child {
  border-bottom: 0;
}

.check-button {
  width: 17px;
  height: 17px;
  flex: none;
  border-radius: 5px;
  border: 1.4px solid #d3d6df;
  background: #fff;
  color: var(--midnight);
  display: grid;
  place-items: center;
  font-size: 11px;
  padding: 0;
}

.check-button.checked {
  background: var(--green);
  border-color: var(--green);
}

.task-copy {
  min-width: 0;
  flex: 1;
}

.task-copy strong {
  display: block;
  font-size: 11px;
  font-weight: 600;
  color: #4d5061;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

.task-copy small {
  display: block;
  font-size: 9px;
  color: #a4a6b1;
  margin-top: 4px;
}

.task-done .task-copy strong {
  text-decoration: line-through;
  color: #aaacb6;
}
```

```
629
630 .priority {
631   font-size: 9px;
632   padding: 4px 7px;
633   border-radius: 12px;
634   white-space: nowrap;
635 }
636
637 .priority.high {
638   background: #fff0ec;
639   color: #d77b64;
640 }
641
642 .priority.medium {
643   background: #fff6e7;
644   color: #c79c56;
645 }
646
647 .priority.low {
648   background: #eff6ec;
649   color: #76a56c;
650 }
651
652 .task-actions {
653   opacity: 0;
654   display: flex;
655   gap: 2px;
656 }
657
658 .task-row:hover .task-actions,.task-row:focus-within .task-actions {
659   opacity: 1;
660 }
661
662 .tiny-action {
663   border: 0;
664   background: none;
665   color: #a0a2ac;
666   padding: 3px;
667   font-size: 13px;
668 }
669
670 .tiny-action:hover {
671   color: #b65b55;
672 }
673
674 .add-inline {
675   margin-top: 11px;
676   border: 0;
677   background: none;
678   padding: 6px 0;
679   color: #8d909d;
680   font-size: 10px;
681 }
682
683 .add-inline:hover {
684   color: var(--midnight);
685 }
686
687 .next-class {
688   display: grid;
689   gap: 11px;
690 }
```

- display: grid or flex arranges child elements. gap controls the space between them.
- :hover responds to a pointer. :focus-within responds when keyboard focus is inside a row.
- A class added or removed by JavaScript can act as a visual state. The CSS does not decide whether the task is complete; the data does.
- Empty-state rules make blank lists look intentional and explain what the user can do next.

The rest of this CSS group styles the next-class card, spending total, budget bar, and recent transaction rows. It also contains the decorative focus card illustration and footer note. JavaScript supplies values for the class and spending previews; the focus illustration is visual decoration created by styled HTML elements.

Lesson 10 - Planner, weekly timetable, and budget styles

The page layouts group related panels into columns. The timetable uses a grid with a time column and one column per weekday. Budget styles cover totals, expense rows, category labels, and filter controls. CSS organizes the content, while app.js supplies the actual schedule and amounts.

Planner columns, week strip, and timetable panel

```
915 /* Planner, timetable, budget, and grade page layouts */
916
917 .planner-layout {
918   display: grid;
919   grid-template-columns: 1.55fr .75fr;
```

```
  gap: 16px;
}

.planner-main {
  padding: 22px;
}

.planner-side {
  padding: 22px;
  background: #fff;
}

.filter-tabs {
  display: flex;
  gap: 5px;
  border-bottom: 1px solid var(--line);
  margin-bottom: 8px;
}

.filter-tab {
  border: 0;
  background: none;
  color: #9699a6;
  padding: 9px 11px;
  font-size: 10px;
  border-bottom: 2px solid transparent;
}

.filter-tab.selected {
  color: var(--midnight);
  border-color: var(--green);
  font-weight: 600;
}

.full-task-list .task-row {
  min-height: 58px;
}

.planner-side h2 {
  font-size: 18px;
  margin: 12px 0 8px;
}

.planner-side .page-subtitle {
  font-size: 11px;
}

.break-card {
  display: flex;
  align-items: center;
  gap: 12px;
  background: #f5f8f1;
  border-radius: 9px;
  padding: 14px;
  margin-top: 22px;
}

.break-card>span {
  color: #7cbd6e;
  font-size: 20px;
}

.break-card strong,.break-card small {
  display: block;
}

.break-card strong {
  font-size: 11px;
  color: #535668;
}

.break-card small {
  font-size: 9px;
  color: #9b9eaa;
  margin-top: 4px;
}

.week-mini {
  margin-top: 24px;
  display: grid;
  grid-template-columns: repeat(7,1fr);
  gap: 4px;
}

.mini-day {
  text-align: center;
  font-size: 8px;
  color: #a4a6b2;
}

.mini-day b {
  display: grid;
  place-items: center;
  height: 26px;
  width: 26px;
  margin: 7px auto 0;
  border-radius: 50%;
  font-size: 9px;
  font-weight: 500;
  color: #707384;
}
```

```
.mini-day.current b {
  background: var(--green);
  color: var(--midnight);
  font-weight: 700;
}

.timetable-panel {
  padding: 22px;
}

.schedule-legend {
  font-size: 10px;
  color: #9295a3;
}

.schedule-legend span {
  display: flex;
  align-items: center;
  gap: 6px;
}

.legend-dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: var(--green);
}

.table-scroll {
  overflow-x: auto;
}

```

Timetable grid and class cells

```
.timetable {
  min-width: 680px;
  display: grid;
  grid-template-columns: 62px repeat(5,1fr);
  border: 1px solid #eff0f4;
  border-radius: 8px;
  overflow: hidden;
}

.time-head,.day-head {
  background: #fafbfc;
  padding: 13px 9px;
  font-size: 9px;
  font-weight: 700;
  letter-spacing: .5px;
  color: #8e91a0;
  text-align: center;
  border-bottom: 1px solid #eff0f4;
}

.time-head {
  border-right: 1px solid #eff0f4;
}

.time-slot {
  min-height: 66px;
  padding: 8px 6px;
  text-align: center;
  font-size: 9px;
  color: #a4a6b1;
  border-right: 1px solid #f0f1f5;
  border-bottom: 1px solid #f0f1f5;
}

.class-slot {
  min-height: 66px;
  padding: 5px;
  border-bottom: 1px solid #f0f1f5;
  border-right: 1px solid #f0f1f5;
  position: relative;
}

.class-block {
  height: 100%;
  min-height: 50px;
  border-radius: 6px;
  background: #edf8e9;
  border-left: 3px solid #8DDA7A;
  padding: 7px 7px;
  color: #4e6b48;
  cursor: pointer;
}

.class-block:hover {
  background: #e1f2db;
}

.class-block strong {
  font-size: 9px;
  display: block;
  line-height: 1.35;
}

```

```
.class-block small {
  font-size: 8px;
  display: block;
  color: #799474;
  margin-top: 4px;
}

.section-hint {
  font-size: 10px;
  color: #9b9daa;
  margin: 13px 3px;
}

.budget-stats {
  display: grid;
  grid-template-columns: repeat(3,1fr);
  gap: 14px;
  margin-bottom: 17px;
}

```

Budget summaries and expense rows

```
.budget-stat {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 10px;
}

.budget-stat>span {
  font-size: 10px;
  color: #9295a3;
}

.budget-stat>strong {
  font: 700 23px Manrope,sans-serif;
  letter-spacing: -.7px;
  color: var(--midnight);
}

.budget-stat small {
  font-size: 10px;
  color: #a2a4ae;
}

.subtle-button {
  border: 0;
  background: none;
  color: #7eae72;
  padding: 0;
  font-size: 10px;
}

.expenses-panel {
  padding: 21px;
}

.select-small {
  border: 1px solid #eceef3;
  border-radius: 7px;
  padding: 8px 10px;
  background: white;
  color: #777a89;
  font-size: 10px;
  outline: none;
}

.expense-row {
  display: grid;
  grid-template-columns: 35px 1fr auto auto;
  gap: 12px;
  align-items: center;
  border-bottom: 1px solid #f0f1f5;
  padding: 12px 1px;
}

.expense-row:last-child {
  border: 0;
}

.expense-icon {
  width: 31px;
  height: 31px;
  border-radius: 9px;
  background: #f2f3f6;
  display: grid;
  place-items: center;
  font-size: 14px;
}

.expense-copy strong,.expense-copy small {
  display: block;
}

.expense-copy strong {
  font-size: 11px;
  color: #545768;
}
```

```
.expense-copy small {
  font-size: 9px;
  color: #a1a3ae;
  margin-top: 4px;
}

.expense-category {
  font-size: 9px;
  color: #9093a0;
  background: #f7f8fa;
  border-radius: 12px;
  padding: 5px 8px;
}

.expense-amount {
  font-size: 11px;
  font-weight: 600;
  color: #4f5263;
  min-width: 60px;
  text-align: right;
}

```

If a class block appears at the wrong time, inspect the JavaScript time matching first. If it appears in the right cell but is too narrow or clipped, inspect the timetable grid rules here.

Lesson 11 - Grade calculator and result panel

The grade CSS styles the result summary and its table. Inputs and selects share sizes, borders, and focus rings. The grade formula is in JavaScript; CSS only formats values such as 85.0% after JavaScript places them in the page.

Grade summary, table, fields, actions, and note

```
.grade-summary {
  border-radius: 12px;
  background: var(--midnight);
  min-height: 150px;
  color: white;
  padding: 24px 29px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 17px;
  overflow: hidden;
  position: relative;
}

.grade-summary .eyebrow {
  color: #a3a5b8;
}

.grade-big {
  font: 700 39px Manrope,sans-serif;
  letter-spacing: -1.5px;
  color: var(--green);
  margin: 6px 0 2px;
}

.grade-big small {
  font-size: 15px;
  margin-left: 3px;
}

.grade-caption {
  margin: 0;
  font-size: 10px;
  color: #b0b2c2;
}

.grade-summary-art {
  width: 135px;
  height: 135px;
  background: #242642;
  border-radius: 50%;
  margin: -40px 20px -60px 0;
  display: grid;
  place-items: center;
  color: #b8a3e2;
  font-size: 47px;
}

.grades-panel {
  padding: 22px;
}

.grade-count {
  font-size: 10px;
  color: #999ba8;
}

.grades-table {
  border-collapse: collapse;
  width: 100%;
```

```
  min-width: 550px;
}

.grades-table th {
  text-align: left;
  padding: 10px 8px;
  border-bottom: 1px solid #eff0f4;
  color: #a2a4af;
  font-size: 8px;
  letter-spacing: 1px;
  font-weight: 700;
}

.grades-table td {
  padding: 13px 8px;
  border-bottom: 1px solid #f0f1f5;
  color: #646778;
  font-size: 10px;
}

.grades-table tr:last-child td {
  border-bottom: 0;
}

.grades-table td:first-child {
  font-weight: 600;
  color: #4c4f60;
}

.grades-table input {
  width: 76px;
  padding: 6px 7px;
  border: 1px solid #e8eaf0;
  border-radius: 6px;
  color: #4d5060;
  font-size: 10px;
  outline: none;
}

.grades-table select {
  width: 100%;
  min-width: 112px;
  padding: 6px 7px;
  border: 1px solid #e8eaf0;
  border-radius: 6px;
  background: #fff;
  color: #4d5060;
  font-size: 10px;
  outline: none;
}

.grades-table input:focus {
  border-color: #8DDA7A;
}

.grades-table select:focus {
  border-color: #8DDA7A;
}

.grade-raw {
  color: #4f5263!important;
  font-weight: 600;
  white-space: nowrap;
}

.grade-actions {
  display: flex;
  justify-content: center;
  gap: 10px;
  margin: 18px 0 4px;
}

.row-action {
  border: 0;
  background: transparent;
  color: #a2a4af;
  font-size: 13px;
}

.row-action:hover {
  color: #cb7164;
}

.grade-note {
  font-size: 9px;
  color: #a0a2ac;
  margin: 12px 7px 0;
}

.empty-state {
  text-align: center;
  color: #a1a3ae;
  padding: 25px 10px;
  font-size: 11px;
}

```

- A table needs readable headers and enough room for inputs. On narrow screens, a wrapper can allow horizontal scrolling.

- A focus style helps keyboard users see which input is active.
- Do not try to change the grade calculation by editing a CSS percentage or width. Change getOverallRawGrade() in app.js.

Lesson 12 - Dialog, toast, tablet and phone rules

The dialog rules style labels, fields, action buttons, and a brief feedback toast. Media queries apply alternate rules when the screen width is small. The phone layout can reposition the sidebar and stack cards. The reduced-motion rule respects a system preference that asks for less animation.

Dialog form and toast styles

```
/* Entry dialog, form fields, and feedback toast */

.dialog-head {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  margin-bottom: 20px;
}

.dialog-head h2 {
  font: 700 20px Manrope,sans-serif;
  color: var(--midnight);
  margin: 7px 0 0;
  letter-spacing: -.5px;
}

.close-dialog {
  border: 0;
  background: none;
  color: #9295a3;
  font-size: 22px;
  line-height: 1;
}

.dialog-field {
  margin-bottom: 13px;
}

.dialog-field label {
  display: block;
  color: #777a89;
  font-size: 10px;
  font-weight: 600;
  margin-bottom: 6px;
}

.dialog-field input,.dialog-field select {
  width: 100%;
  height: 38px;
  border: 1px solid #e8eaf0;
  border-radius: 7px;
  padding: 0 10px;
  color: #4b4e5f;
  background: #fff;
  outline: none;
  font-size: 11px;
}

.dialog-field input:focus,.dialog-field select:focus {
  border-color: #8DDA7A;
  box-shadow: 0 0 0 3px #8dda7a25;
}

.dialog-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 11px;
}

.dialog-actions {
  display: flex;
  justify-content: flex-end;
  gap: 8px;
  margin-top: 20px;
}

.dialog-actions .button {
  height: 36px;
}

dialog {
  width: min(420px,calc(100% - 30px));
  border: 1px solid #eceef3;
  border-radius: 13px;
  padding: 23px 24px;
  box-shadow: 0 25px 80px #17182f35;
}

dialog::backdrop {
  background: #17182f65;
  backdrop-filter: blur(2px);
```

```
}

.toast {
  position: fixed;
  bottom: 23px;
  left: 50%;
  transform: translate(-50%,15px);
  padding: 10px 15px;
  border-radius: 8px;
  background: var(--midnight);
  color: white;
  font-size: 11px;
  opacity: 0;
  pointer-events: none;
  transition: .22s;
  z-index: 40;
  box-shadow: 0 6px 20px #17182f30;
}

.toast.show {
  opacity: 1;
  transform: translate(-50%,0);
}

```

Tablet breakpoints

```
/* Tablet adjustments */

@media(max-width:1050px) {
  .content-wrap {
    padding: 30px 27px 18px;
  }

  .topbar {
    padding: 0 27px;
  }

  .sidebar {
    width: 220px;
  }

  .main-area {
    margin-left: 220px;
  }

  .dashboard-grid {
    gap: 12px;
  }

  .focus-copy {
    max-width: 82%;
  }

}

@media(max-width:800px) {
  .sidebar {
    transform: translateX(-100%);
    transition: transform .25s;
    width: 245px;
    box-shadow: 10px 0 40px #17182f22;
  }

  .sidebar.open {
    transform: translateX(0);
  }

  .main-area {
    margin-left: 0;
  }

  .mobile-menu {
    display: block;
    padding: 4px 8px 4px 0;
  }

  .topbar {
    padding: 0 20px;
  }

  .breadcrumb {
    margin-right: auto;
  }

  .content-wrap {
    padding: 27px 20px 18px;
  }

  /* Tablet planner and timetable layout */

  .planner-layout {
    grid-template-columns: 1fr .72fr;
  }

  .dashboard-grid {
    grid-template-columns: 1fr 1fr;
  }

```

```
  .welcome-row h1 {
    font-size: 26px;
  }

}

```

Phone rules and reduced motion

```
/* Phone adjustments */

@media(max-width:600px) {
  .topbar {
    height: 59px;
    padding: 0 16px;
  }

  .topbar-right {
    gap: 10px;
  }

  .today-chip {
    font-size: 0;
  }

  .today-chip .live-dot {
    font-size: initial;
  }

  .content-wrap {
    padding: 24px 15px 15px;
  }

  .welcome-row {
    align-items: flex-start;
    gap: 12px;
    margin-bottom: 21px;
  }

  .welcome-row h1 {
    font-size: 23px;
  }

  .page-subtitle {
    font-size: 11px;
    max-width: 250px;
  }

  .welcome-row>.button {
    white-space: nowrap;
    padding: 0 10px;
    height: 36px;
    font-size: 10px;
    margin-top: 11px;
  }

  /* Compact dashboard summary cards */

  .stats-grid {
    grid-template-columns: 1fr;
    gap: 9px;
  }

  .stat-card {
    padding: 14px 16px;
  }

  .stat-number {
    font-size: 22px;
    margin: 5px 0;
  }

  .stat-icon {
    width: 27px;
    height: 27px;
  }

  .stat-foot {
    margin-top: 5px;
  }

  .progress-track {
    margin: 8px 0 5px;
  }

  .dashboard-grid {
    grid-template-columns: 1fr;
    gap: 10px;
  }

  .panel {
    padding: 16px;
  }

  .focus-panel {
    min-height: 160px;
  }

  /* Phone planner, budget, and grade layout */
```

```
1665
1666    .planner-layout {
1667      grid-template-columns: 1fr;
1668    }
1669
1670    .planner-side {
1671      order: -1;
1672    }
1673
1674    .planner-side .week-mini {
1675      margin-top: 15px;
1676    }
1677
1678    .budget-stats {
1679      grid-template-columns: 1fr;
1680      gap: 9px;
1681    }
1682
1683    .budget-stat {
1684      display: grid;
1685      grid-template-columns: 1fr auto;
1686      gap: 7px 10px;
1687      align-items: center;
1688    }
1689
1690    .budget-stat>span {
1691      grid-column: 1;
1692    }
1693
1694    .budget-stat>strong {
1695      grid-column: 1;
1696      grid-row: 2;
1697    }
1698
1699    .budget-stat .subtle-button,.budget-stat small {
1700      grid-column: 2;
1701      grid-row: 2;
1702    }
1703
1704    .expenses-panel {
1705      padding: 15px;
1706    }
1707
1708    .expense-row {
1709      grid-template-columns: 32px 1fr auto;
1710      gap: 8px;
1711    }
1712
1713    .expense-category {
1714      display: none;
1715    }
1716
1717    .expense-amount {
1718      min-width: 54px;
1719    }
1720
1721    .grade-summary {
1722      padding: 20px;
1723      min-height: 135px;
1724    }
1725
1726    .grade-summary-art {
1727      margin-right: -35px;
1728    }
1729
1730    .grades-panel {
1731      padding: 16px;
1732    }
1733
1734    .grades-table-wrap {
1735      overflow-x: auto;
1736    }
1737
1738    .footer-note {
1739      line-height: 1.7;
1740    }
1741
1742    .dialog-grid {
1743      grid-template-columns: 1fr 1fr;
1744    }
1745
1746  }
1747
1748  @media(prefers-reduced-motion:reduce) {
1749    *,*:before,*:after {
1750      scroll-behavior: auto!important;
1751      animation-duration: .01ms!important;
1752      transition-duration: .01ms!important;
1753    }
1754
1755  }
```

- @media (max-width: 800px) means these rules apply at 800 pixels wide or less.
- A later matching rule can override a property from an earlier desktop rule.
- Check desktop, phone, and reduced-motion settings after a layout or animation edit. A desktop change can overflow a phone, and prefers-reduced-motion lets users avoid nonessential movement.

PART 4 / JavaScript: data, rendering, and behavior

JavaScript is the behavior layer. In Plannex, it starts with sample records, loads saved records when available, renders each page, responds to events, and saves updates. The file is wrapped in a function that runs immediately, keeping its variables private to this app.

Lesson 13 - The app wrapper, settings, and sample data

The opening (() => { ... })(); form is an immediately invoked function expression. It creates a private workspace so names such as state or render do not become general browser globals. The first section defines settings such as the storage key, timetable days and hours, grade categories, expense icons, current date, and seed data.

The wrapper, settings, and the beginning of sample data

```
(() => {
  // 1. Starter data and settings
  // These values set the first-run sample content and the timetable/grade options.
  const KEY = "plannex-student-planner-v1";
  const DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri"];
  const HOURS = [
    "8:00",
    "9:00",
    "10:00",
    "11:00",
    "12:00",
    "1:00",
    "2:00",
    "3:00",
    "4:00",
  ];
  const GRADE_CATEGORIES = ["Quiz", "Long Quiz", "Assignment", "Exam"];
  const seed = {
    allowance: 12000,
    tasks: [
      {
        id: "t1",
        title: "Finish biology lab notes",
        due: "Today · 11:30 AM",
        priority: "High",
        done: false,
      },
      {
        id: "t2",
        title: "Read chapter 4 for literature",
        due: "Today · 2:00 PM",
        priority: "Medium",
        done: false,
      },
      {
        id: "t3",
        title: "Submit statistics problem set",
        due: "Today · 5:00 PM",
        priority: "High",
        done: true,
      },
      {
        id: "t4",
        title: "Pack things for tomorrow",
        due: "Today · Anytime",
```

Each task, class, expense, or grade is one object. A collection such as tasks is an array of objects. The id gives each record a stable identity, so the app can toggle or remove exactly one row. The seed object is the starter set shown when the app has no saved state.

Record	Typical fields	Why the fields matter
Task	id, title, due, priority, done	done drives completion counts and task styling.
Class	id, day, time, name, room	day and time place the class on the weekly grid.
Expense	id, name, amount, category, date	amount contributes to spending; category drives filtering.
Grade row	id, category, score, total	score and total feed the raw percentage calculation.

- Change the seed object when you want different demo content on a clean first run.
- Changing the seed does not overwrite data already saved in your browser. Use Restore sample data to copy the new seed into the current state.

- Keep the names used by the render and dialog functions consistent. Renaming a field in only one place breaks that feature.

Lesson 14 - Saved state and small helper functions

The state variable is the app's working copy of all planner data. loadState() reads a saved JSON string and turns it back into objects. persist() performs the reverse: JSON.stringify() converts state to text and localStorage.setItem() stores it in this browser. The helpers below reduce repetition across the rest of the file.

Saved-state loader, DOM shortcuts, formatting, ids, dates, and toast

```
// 2. Saved state and small helpers
// Short DOM helpers keep the render functions readable.
const $ = (s, root = document) => root.querySelector(s);
const $$ = (s, root = document) => [...root.querySelectorAll(s)];
let state;
try {
  state = JSON.parse(localStorage.getItem(KEY)) || structuredClone(seed);
  // Keep old subject grades in storage before switching to assessment rows.
  if (state.grades?.some((grade) => !("category" in grade))) {
    state.previousSubjectGrades = structuredClone(state.grades);
    state.grades = structuredClone(seed.grades);
    state.gradesComputed = false;
  }
  if (typeof state.gradesComputed !== "boolean") state.gradesComputed = false;
} catch {
  state = structuredClone(seed);
}
// Save the complete planner snapshot after each change so the sections stay in sync.
const persist = () => localStorage.setItem(KEY, JSON.stringify(state));
const money = (value) =>
  new Intl.NumberFormat("en-PH", {
    style: "currency",
    currency: "PHP",
    maximumFractionDigits: 0,
  }).format(Number(value) || 0);
const EXPENSE_ICONS = {
  Food: " ",
  Transport: " ",
  School: " ",
  Personal: " ",
  Other: "·",
};
const escapeHtml = (value) =>
  String(value).replace(
    /[&<>"']/g,
    (c) =>
      ({
        "&": "&amp;",
        "<": "&lt;",
        ">": "&gt;",
        '"': "&quot;",
        "'": "&#39;",
      })[c],
  );
const makeId = () =>
  `${Date.now().toString(36)}${Math.random().toString(36).slice(2, 7)}`;
const today = new Date();
const friendlyDate = today.toLocaleDateString("en-PH", {
  weekday: "long",
  month: "long",
  day: "numeric",
});
$("#today-label").textContent = friendlyDate;
$("#planner-date").textContent = `TODAY · ${friendlyDate.toUpperCase()}`;
$("#greeting").textContent =
  `${today.getHours() < 12 ? "GOOD MORNING" : today.getHours() < 18 ? "GOOD AFTERNOON" : "GOOD EVENING"}, JAMIE`;
const toast = (message) => {
  const node = $("#toast");
  node.textContent = message;
  node.classList.add("show");
  clearTimeout(toast.timer);
  toast.timer = setTimeout(() => node.classList.remove("show"), 2200);
};

```

Helper	Plain meaning	Why it is reused
$ and $$	Find one element or a list of elements.	Keeps DOM selection short and readable.
loadState()	Restore saved values or start from sample data.	Handles first visit and older saved data.
persist()	Save the current state in local storage.	Called after edits and after a render.
money()	Format a number as Philippine pesos.	Keeps money displays consistent.
escapeHtml()	Escape special characters before placing user text in HTML.	Prevents typed text from being treated as markup.

Helper	Plain meaning	Why it is reused
makeId()	Create an identifier for a new row.	Lets later actions target the exact record.
toast()	Show temporary on-screen feedback.	Confirms small actions such as deleting a row.

localStorage belongs to the browser profile on this device. It is not a cloud account or a shared database. Clearing browser site data can remove the saved planner state.

Lesson 15 - Render tasks and show progress

Rendering means turning current data into page content. tasksMarkup() builds the HTML for one task row and handles the empty list message. renderTasks() calculates the remaining and completed counts, sets the progress bar width, and places task markup in both the Overview and planner list. The same tasks can appear in two places without duplicating their data.

Task markup and task counts

```
247   // 3. Rendering helpers
248   // These functions turn saved state into the cards, tables, and totals on screen.
249   // Shared markup keeps task rows consistent on the dashboard and full planner page.
250   function tasksMarkup(tasks, controls = true) {
251     if (!tasks.length)
252       return '<div class="empty-state">Nothing here just yet. Add a task when you’re ready.</div>';
253     return tasks
254       .map(
255         (task) =>
256           `<div class="task-row ${task.done ? "task-done" : ""}"><button class="check-button ${task.done ? "checked" : ""}" data-toggle-task="${t
    ask.id}" aria-label="${task.done ? "Mark incomplete" : "Mark complete"}">${task.done ? " " : ""}</button><div class="task-copy"><strong>${escapeH
    tml(task.title)}</strong><small>${escapeHtml(task.due || "Today")}</small></div><span class="priority ${(task.priority || "Medium").toLowerCase()
    }">${escapeHtml(task.priority || "Medium")}</span>${controls ? `<span class="task-actions"><button class="tiny-action" data-delete-task="${task.i
    d}" aria-label="Delete task">×</button></span>` : ""}</div>`,
257       )
258       .join("");
259   }
260   function getRowGrade(row) {
261     if (row.score === "" || row.score == null || row.total === "" || row.total == null)
262       return null;
263     const score = Number(row.score);
264     const total = Number(row.total);
265     return Number.isFinite(score) && Number.isFinite(total) && total > 0
266       ? (score / total) * 100
267       : null;
268   }
269   // Calculate a raw grade from points, so items with larger totals naturally count more.
270   function getOverallRawGrade() {
271     const valid = state.grades.filter((row) => getRowGrade(row) !== null);
272     if (!valid.length) return null;
273     const earned = valid.reduce((sum, row) => sum + Number(row.score), 0);
274     const possible = valid.reduce((sum, row) => sum + Number(row.total), 0);
275     return possible > 0 ? (earned / possible) * 100 : null;
276   }
277   function renderTasks() {
278     const remaining = state.tasks.filter((t) => !t.done).length,
279       done = state.tasks.length - remaining;
280     $("#stat-tasks").innerHTML =
281       `${remaining} <small>${remaining === 1 ? "task" : "tasks"} left</small>`;
282     $("#today-count").textContent = state.tasks.length;
283     $("#planner-count").textContent = remaining;
284     $("#task-foot").textContent = state.tasks.length
285       ? `${done} of ${state.tasks.length} completed`
286       : "Add a task to get started";
287     $("#task-progress").style.width =
288       `${state.tasks.length ? (done / state.tasks.length) * 100 : 0}%`;
289     $("#overview-tasks").innerHTML = tasksMarkup(state.tasks.slice(0, 4));
290     const filter = $(".filter-tab.selected")?.dataset.filter || "all";
291     $("#planner-tasks").innerHTML = tasksMarkup(
292       state.tasks.filter(
293         (t) => filter === "all" || (filter === "open" ? !t.done : t.done),
294       ),
295     );
296   }
```

- filter() selects records that pass a condition. Here it finds unfinished tasks or tasks matching the selected tab.
- slice(0, 4) makes the Overview show only a short preview. The full planner still uses the complete array.
- innerHTML is used where JavaScript generates child markup. User-provided text is escaped with escapeHtml().
- textContent is used for plain text labels because it does not parse the value as HTML.
- The task progress width is completed count divided by all tasks, multiplied by 100. When there are no tasks, the width is set to zero.

Lesson 16 - Budget totals, expenses, and filters

renderBudget() totals expense amounts, subtracts them from the allowance, updates summary cards, and calculates the budget bar. renderExpenses() reads the selected filter and draws only expenses in that category. Both functions derive display values from state instead of storing a second copy of the totals.

Budget totals and filtered expense rows

```
297   function renderBudget() {
298     const spent = state.expenses.reduce((sum, e) => sum + Number(e.amount), 0),
299       remaining = Number(state.allowance) - spent,
300       ratio =
301         Number(state.allowance) > 0
302           ? Math.min(100, (spent / Number(state.allowance)) * 100)
303           : 100;
304     $("#stat-budget").textContent = money(remaining);
305     $("#budget-foot").textContent =
306       `of your ${money(state.allowance)} monthly allowance`;
307     $("#overview-spent").textContent = money(spent);
308     $("#spent-label").textContent = `${money(spent)} spent`;
309     $("#limit-label").textContent = `${money(state.allowance)} limit`;
310     $("#budget-bar").style.width = `${ratio}%`;
311     $("#budget-total").textContent = money(state.allowance);
312     $("#budget-spent").textContent = money(spent);
313     $("#budget-remaining").textContent = money(remaining);
314     $("#budget-remaining").style.color = remaining < 0 ? "#ce8065" : "";
315     $("#budget-remaining-note").textContent =
316       remaining < 0 ? "Over your monthly allowance" : "You’re doing great";
317     const icons = EXPENSE_ICONS;
318     $("#recent-transactions").innerHTML =
319       state.expenses
320         .slice(0, 2)
321         .map(
322           (e) =>
323             `<div class="transaction-row"><span class="transaction-icon">${icons[e.category] || "·"}</span><span class="transaction-info"><strong
    >${escapeHtml(e.name)}</strong><small>${escapeHtml(e.category)}</small></span><span class="transaction-amount">−${money(e.amount)}</span></div>`,
324         )
325         .join("") || '<div class="empty-state">No expenses yet.</div>';
326     renderExpenses();
327   }
328   function renderExpenses() {
329     const filter = $("#expense-filter").value;
330     const rows = state.expenses.filter(
331       (e) => filter === "all" || e.category === filter,
332     );
333     const icons = EXPENSE_ICONS;
334     $("#expense-list").innerHTML = rows.length
335       ? rows
336           .map(
337             (e) =>
338               `<div class="expense-row"><span class="expense-icon">${icons[e.category] || "·"}</span><span class="expense-copy"><strong>${escapeH
    tml(e.name)}</strong><small>${new Date(`${e.date}T12:00:00`).toLocaleDateString("en-PH", { month: "short", day: "numeric", year: "numeric" })}</s
    mall></span><span class="expense-category">${escapeHtml(e.category)}</span><span class="expense-amount">−${money(e.amount)}</span></div>`,
339           )
340           .join("")
341       : '<div class="empty-state">No expenses in this category.</div>';
342   }
```

For example, if the allowance is ₱12,000 and recorded expenses total ₱3,450, the remaining amount is ₱8,550. The bar uses spent divided by allowance, capped at 100 percent for display. The remaining amount itself can still go below zero if spending exceeds the allowance.

- reduce() accumulates a sum across an array.
- The expense filter is independent from the total: totals include all recorded expenses even when the list is filtered.
- An empty array renders a helpful message instead of leaving a blank box.

Lesson 17 - Grade rows and raw grade calculation

getRowGrade() calculates an individual row as score divided by total, multiplied by 100. getOverallRawGrade() adds all valid earned points and divides by all possible points. This is a points-based raw grade, so an assessment worth more points contributes more to the result.

The grade formula and validation

```
260   function getRowGrade(row) {
261     if (row.score === "" || row.score == null || row.total === "" || row.total == null)
262       return null;
263     const score = Number(row.score);
264     const total = Number(row.total);
265     return Number.isFinite(score) && Number.isFinite(total) && total > 0
266       ? (score / total) * 100
267       : null;
268   }
269   // Calculate a raw grade from points, so items with larger totals naturally count more.
```

```
function getOverallRawGrade() {
  const valid = state.grades.filter((row) => getRowGrade(row) !== null);
  if (!valid.length) return null;
  const earned = valid.reduce((sum, row) => sum + Number(row.score), 0);
  const possible = valid.reduce((sum, row) => sum + Number(row.total), 0);
  return possible > 0 ? (earned / possible) * 100 : null;
}
```

Example: a score of 18/20 and a score of 45/60 produce (18 + 45) / (20 + 60) x 100 = 78.75%. This is different from averaging 90% and 75% equally, which would give 82.5%. The app waits for Compute grade before showing a new average after score edits.

The summary message and the editable grade rows

```
function renderGradeSummary() {
  const avg = state.gradesComputed ? getOverallRawGrade() : null;
  $("#stat-grade").innerHTML =
    avg === null
      ? "— <small>%</small>"
      : `${avg.toFixed(1)}<small>%</small>`;
  $("#grade-average").innerHTML =
    avg === null ? "—<small>%</small>" : `${avg.toFixed(1)}<small>%</small>`;
  $("#grade-caption").textContent =
    avg === null
      ? "Enter scores and totals, then compute your grade."
      : avg >= 90
        ? "Excellent work — your effort is showing."
        : avg >= 75
          ? "You're making steady progress. Keep going."
          : "Use this result to guide your next study session.";
  $("#grade-count").textContent =
    `${state.grades.length} ${state.grades.length === 1 ? "row" : "rows"}`;
}
function renderGrades() {
  renderGradeSummary();
  $("#grade-rows").innerHTML = state.grades.length
    ? state.grades
        .map(
          (row) => {
            const rawGrade = getRowGrade(row);
            return `<tr><td><select data-grade-row="${row.id}" data-grade-field="category" aria-label="Assessment category">${GRADE_CATEGORIES.
map((category) => `<option value="${category}" ${row.category === category ? "selected" : ""}>${category}</option>`).join("")}</select></td><td><
input type="number" min="0" step="any" value="${row.score ?? ""}" placeholder="Score" data-grade-row="${row.id}" data-grade-field="score" aria-la
bel="Score"></td><td><input type="number" min="0.01" step="any" value="${row.total ?? ""}" placeholder="Total" data-grade-row="${row.id}" data-gr
ade-field="total" aria-label="Total possible points"></td><td class="grade-raw">${state.gradesComputed && rawGrade !== null ? `${rawGrade.toFixed
(1)}%` : "—"}</td><td><button class="row-action" data-delete-grade="${row.id}" aria-label="Remove grade row">×</button></td></tr>`;
          },
        )
        .join("")
    : '<tr><td colspan="5"><div class="empty-state">No grade rows yet. Add a row to get started.</div></td></tr>';
}
// The timetable uses 1:00–4:00 for afternoon classes; normalize those for comparisons.
```

- Blank rows are skipped because there is no complete score/total pair to calculate.
- A total must be greater than zero. A zero or blank denominator cannot produce a useful percentage.
- Number.isFinite() rejects invalid values such as NaN or Infinity.
- The grade categories are labels. They are not weighted differently in this formula.
- If the course syllabus assigns category weights, the formula must be intentionally redesigned to use those weights.

The displayed result is a raw points percentage, not a guaranteed official course grade. Always compare it with your instructor's grading rules.

Lesson 18 - Class timetable and week strip

renderClasses() generates the time column and weekday headings, loops over the HOURS and DAYS arrays, looks for a matching class, then inserts a class block or empty cell. It also finds the next class for today. toMinutes() converts the supported afternoon hours to a comparable 24-hour number. renderWeek() calculates the Monday-through-Sunday date strip around today.

Time conversion, weekly grid, next class, mini week, and render()

```
function toMinutes(time) {
  const [hour, minute] = time.split(":").map(Number);
  const normalized = hour >= 1 && hour <= 4 ? hour + 12 : hour;
  return normalized * 60 + (minute || 0);
}
function renderClasses() {
  const grid = $("#timetable");
  grid.innerHTML = `<div class="time-head">TIME</div>${DAYS.map((d) => `<div class="day-head">${d.toUpperCase()}</div>`).join("")}`;
  for (const hour of HOURS) {
    grid.insertAdjacentHTML(
```

```
386           "beforeend",
387           `<div class="time-slot">${hour}</div>`,
388         );
389         for (const day of DAYS) {
390           const item = state.classes.find(
391             (c) => c.day === day && parseInt(c.time, 10) === parseInt(hour, 10),
392           );
393           grid.insertAdjacentHTML(
394             "beforeend",
395             `<div class="class-slot">${item ? `<div class="class-block" title="Click to remove this class" data-delete-class="${item.id}"><strong>${escapeHtml(item.name)}</strong><small>${escapeHtml(item.room)}</small></div>` : ""}</div>`,
396           );
397         }
398       }
399       const todayCode = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"][
400         today.getDay()
401       ];
402       const now = today.getHours() * 60 + today.getMinutes();
403       const upcoming = state.classes
404         .filter((c) => c.day === todayCode && toMinutes(c.time) >= now)
405         .sort((a, b) => toMinutes(a.time) - toMinutes(b.time))[0];
406       const next = $("#next-class");
407       next.innerHTML = upcoming
408         ? `<div class="class-card"><span class="class-time">${escapeHtml(upcoming.time)}</span><span class="class-stripe"></span><span class="class-info"><strong>${escapeHtml(upcoming.name)}</strong><small>${escapeHtml(upcoming.room)}</small></span></div><div class="no-class">Your next class today.</div>`
409         : `<div class="no-class">${DAYS.includes(todayCode) ? "No more classes today. Take a breather." : "No classes scheduled today. Enjoy your day."}</div>`;
410     }
411     function renderWeek() {
412       const start = new Date(today);
413       start.setDate(today.getDate() - ((today.getDay() + 6) % 7));
414       $("#week-mini").innerHTML = Array.from({ length: 7 }, (_, i) => {
415         const date = new Date(start);
416         date.setDate(start.getDate() + i);
417         return `<div class="mini-day ${i === (today.getDay() + 6) % 7 ? "current" : ""}">${["M", "T", "W", "T", "F", "S", "S"][i]}<b>${date.getDate()}</b></div>`;
418       }).join("");
419     }
420     function render() {
421       renderTasks();
422       renderBudget();
423       renderGrades();
424       renderClasses();
425       renderWeek();
426       persist();
427     }
428
```

- Nested loops create a grid: for each time slot, create one cell for each day.
- find() returns the first class whose day and start hour match the grid position.
- The timetable stores start times but not class lengths, so each class appears in a single time cell.
- Render() calls all display functions in one place, then persists the state. After changing app data, call render() to refresh all views.

Lesson 19 - Navigation and event handling

navigate(page) checks that a page exists, toggles visibility classes, highlights the matching menu link, updates the breadcrumb, writes the location hash, and closes the mobile menu. The event listeners below handle navigation, the phone menu, task actions, deletion, filtering, expense changes, and grade edits.

Navigation and delegated task, grade, and class actions

```
429   // 4. Navigation and page interactions
430   function navigate(page) {
431     if (!$("#page-" + page)) page = "overview";
432     $$(".page-section").forEach((n) =>
433       n.classList.toggle("active-page", n.id === `page-${page}`),
434     );
435     $$(".nav-link").forEach((n) =>
436       n.classList.toggle("active", n.dataset.page === page),
437     );
438     $("#crumb-current").textContent = {
439       overview: "Overview",
440       planner: "Daily planner",
441       classes: "Class schedule",
442       budget: "My budget",
443       grades: "Grade calculator",
444     }[page];
445     history.replaceState(null, "", `#${page}`);
446     $("#sidebar").classList.remove("open");
447   }
448   $$(".nav-link").forEach((link) =>
449     link.addEventListener("click", (e) => {
450       e.preventDefault();
451       navigate(link.dataset.page);
452     }),
453   );
454   $("#menu-toggle").addEventListener("click", () =>
```

```
    $("#sidebar").classList.toggle("open"),
  );
  // Delegate clicks because task, class, and grade controls are rebuilt during rendering.
  document.addEventListener("click", (e) => {
    const target = e.target.closest("[data-action]");
    if (target) openDialog(target.dataset.action);
    const toggle = e.target.closest("[data-toggle-task]");
    if (toggle) {
      const task = state.tasks.find((t) => t.id === toggle.dataset.toggleTask);
      if (task) task.done = !task.done;
      render();
    }
    const del = e.target.closest("[data-delete-task]");
    if (del) {
      state.tasks = state.tasks.filter((t) => t.id !== del.dataset.deleteTask);
      render();
      toast("Task removed");
    }
    const gradeDel = e.target.closest("[data-delete-grade]");
    if (gradeDel) {
      state.grades = state.grades.filter(
        (g) => g.id !== gradeDel.dataset.deleteGrade,
      );
      state.gradesComputed = false;
      render();
      toast("Grade row removed");
    }
    const classDel = e.target.closest("[data-delete-class]");
    if (classDel) {
      state.classes = state.classes.filter(
        (c) => c.id !== classDel.dataset.deleteClass,
      );
      render();
      toast("Class removed");
    }
  });
```

Filters and grade input interactions

```
  $$(".filter-tab").forEach((tab) =>
    tab.addEventListener("click", () => {
      $$(".filter-tab").forEach((t) =>
        t.classList.toggle("selected", t === tab),
      );
      renderTasks();
    }),
  );
  $("#expense-filter").addEventListener("change", renderExpenses);
  $("#grade-rows").addEventListener("change", (e) => {
    const field = e.target.closest("[data-grade-row]");
    if (!field) return;
    const row = state.grades.find((g) => g.id === field.dataset.gradeRow);
    if (row) {
      const property = field.dataset.gradeField;
      row[property] = property === "category"
        ? field.value
        : field.value === ""
          ? ""
          : Math.max(0, Number(field.value) || 0);
      if (property !== "category") {
        state.gradesComputed = false;
        $$(".grade-raw").forEach((cell) => (cell.textContent = "—"));
      }
      renderGradeSummary();
      persist();
    }
  });
  $("#add-grade-row").addEventListener("click", () => {
    state.grades.push({
      id: makeId(),
      category: GRADE_CATEGORIES[0],
      score: "",
      total: "",
    });
    state.gradesComputed = false;
    renderGrades();
    persist();
    $('#grade-rows tr:last-child [data-grade-field="score"]')?.focus();
  });
  $("#compute-grade").addEventListener("click", () => {
    state.gradesComputed = true;
    renderGrades();
    persist();
  });

```

Why delegated clicks are used

Many task and class buttons are created again every time render() runs. A listener attached once to each row would disappear when the rows are replaced. Instead, one listener is attached to document. The event bubbles upward; closest() checks whether the clicked element or one of its parents matches a data attribute such as data-toggle-task. This is event delegation.

- e is the event object. e.target is the element that was actually clicked.
- dataset reads data-* attributes. data-delete-task becomes dataset.deleteTask.
- The grade change listener finds the matching saved row by id, updates one property, clears stale computed results, then saves.
- A filter event can rerender only the relevant list instead of rebuilding the whole screen.
- The hash lets a page be opened with a URL ending in #budget or another known page.

Lesson 20 - Dialog fields, save, cancel, reset, and startup

field() creates a labeled text/number input or a dropdown. openDialog(action) selects one configuration for adding a task, expense, class, or editing the allowance. The configuration supplies a title, fields, and a save callback. One form submit handler validates and commits the chosen action. Cancel and close only close the dialog and never call the save callback.

Dialog controls, field builder, action configurations, and Save handling

```
  // 5. Shared entry dialog
  // Cancel and close only dismiss the dialog; the form submit handler is reserved for Save.
  const dialog = $("#entry-dialog"),
    form = $("#entry-form");
  document.addEventListener("click", (e) => {
    if (e.target.closest("[data-close-dialog]")) dialog.close();
  });
  function field(
    label,
    name,
    type = "text",
    value = "",
    options = null,
    extra = "",
  ) {
    if (options)
      return `<div class="dialog-field"><label for="field-${name}">${label}</label><select id="field-${name}" name="${name}" ${extra}>${options.map((o) => `<option value="${escapeHtml(o)}" ${o === value ? "selected" : ""}>${escapeHtml(o)}</option>`).join("")}</select></div>`;
    return `<div class="dialog-field"><label for="field-${name}">${label}</label><input id="field-${name}" name="${name}" type="${type}" value="${escapeHtml(value)}" ${type === "number" ? 'min="0" step="1"' : ""} ${extra} required></div>`;
  }
  // One dialog builder supplies the fields and save action for each add/edit interaction.
  function openDialog(action) {
    const config = {
      "add-task": {
        title: "Add a task",
        subtitle: "A SMALL THING, DONE WELL",
        fields: `${field("What needs doing?", "title", "text", "", "", 'placeholder="e.g. Review lecture notes"')}<div class="dialog-grid">${field("Due", "due", "text", "Today · Anytime", "", 'placeholder="Today · 3:00 PM"')}${field("Priority", "priority", "text", "Medium", ["Low", "Medium", "High"])}</div>`,
        save: (data) =>
          state.tasks.unshift({
            id: makeId(),
            title: data.get("title").trim(),
            due: data.get("due").trim() || "Today",
            priority: data.get("priority"),
            done: false,
          }),
      },
      "add-expense": {
        title: "Add an expense",
        subtitle: "MONEY CHECK-IN",
        fields: `${field("What did you spend on?", "name", "text", "", "", 'placeholder="e.g. Lunch"')}<div class="dialog-grid">${field("Amount (₱)", "amount", "number", "", "", 'min="1"')}${field("Category", "category", "text", "Food", ["Food", "Transport", "School", "Personal", "Other"])}</div>${field("Date", "date", "date", today.toISOString().slice(0, 10))}`,
        save: (data) =>
          state.expenses.unshift({
            id: makeId(),
            name: data.get("name").trim(),
            amount: Number(data.get("amount")),
            category: data.get("category"),
            date: data.get("date"),
          }),
      },
      "add-class": {
        title: "Add a class",
        subtitle: "BUILD YOUR WEEK",
        fields: `${field("Subject", "name", "text", "", "", 'placeholder="e.g. Chemistry"')}<div class="dialog-grid">${field("Day", "day", "text", "Mon", DAYS)}${field("Start time", "time", "text", "9:00", ["8:00", "9:00", "10:00", "11:00", "12:00", "1:00", "2:00", "3:00", "4:00"])}</div>${field("Room or location", "room", "text", "", "", 'placeholder="e.g. Science Hall · 204"')}`,
        save: (data) =>
          state.classes.push({
            id: makeId(),
            day: data.get("day"),
            time: data.get("time"),
            name: data.get("name").trim(),
            room: data.get("room").trim() || "Room not set",
          }),
      },
      "edit-budget": {
        title: "Monthly allowance",
        subtitle: "SET YOUR MONTHLY PLAN",
```

```
        fields: field(
          "Allowance (₱)",
          "allowance",
          "number",
          state.allowance,
          "",
          'min="0"',
        ),
        save: (data) => (state.allowance = Number(data.get("allowance"))),
      },
    }[action];
    if (!config) return;
    $("#dialog-title").textContent = config.title;
    $("#dialog-eyebrow").textContent = config.subtitle;
    $("#dialog-fields").innerHTML = config.fields;
    form.onsubmit = (e) => {
      e.preventDefault();
      if (!form.reportValidity()) return;
      const data = new FormData(form);
      config.save(data);
      render();
      dialog.close();
      toast(
        action === "edit-budget"
          ? "Allowance updated"
          : "Added to your planner",
      );
    };
    dialog.showModal();
    setTimeout(() => $("#dialog-fields input")?.focus(), 30);
  }

```

The final section asks before replacing the state with seed data, then navigates to the page named by the URL hash if it is valid, and calls render() to build the first screen.

Restore sample data and initialize the app

```
  // 6. Reset and first page load
  $("#reset-data").addEventListener("click", () => {
    if (
      confirm(
        "Restore the original sample planner? Your current changes will be replaced.",
      )
    ) {
      state = structuredClone(seed);
      render();
      navigate("overview");
      toast("Sample planner restored");
    }
  });
  const initial = location.hash.replace("#", "");
  navigate(
    ["overview", "planner", "classes", "budget", "grades"].includes(initial)
      ? initial
      : "overview",
  );
  render();
})();
```

- FormData(form) reads submitted values by their name attributes.
- reportValidity() asks the browser to enforce required fields and input constraints.
- e.preventDefault() stops normal form navigation so the app can save without reloading.
- dialog.close() closes the native dialog. Since Cancel does not submit, no state changes.
- structuredClone(seed) creates a separate copy so edits do not mutate the original sample data.
- location.hash is checked against the known page names to avoid navigating to a missing page.

To add another dialog action, add a button with a data-action value in HTML, then add a matching configuration in openDialog(). The action string must match exactly.

PART 5 / Practice edits and troubleshooting

The best way to learn is to make one small change, see what it does, and undo it. Make a copy of the project first. Keep one change at a time so you know which edit caused a problem.

Practice 1 - Change a brand color

Open styles.css and change the value of --green under :root. Save and refresh the page. Buttons, highlights, and progress accents that use the token should change together.

Practice 2 - Change the starter allowance

Open app.js and find allowance inside seed. Change the number. Existing browser data remains in local storage, so click Restore sample data if you want to see the updated starter value immediately.

Practice 3 - Add a task to the sample list

In the seed tasks array, copy one task object and change its id, title, due text, and priority. Keep commas between objects and keep the value of done as true or false. Restore sample data to load the edited starter list into the browser.

Practice 4 - Change grade category choices

Edit GRADE_CATEGORIES near the top of app.js. The dropdown options in generated rows use this array. Older saved rows may still hold a category that is no longer in the list; avoid removing categories if you need to preserve existing sample entries.

Practice 5 - Change text or a page label

For visible text already in the layout, edit the words in index.html. For text created from JavaScript, find the matching render or dialog function in app.js. Do not rename ids unless you also update every JavaScript selector that uses them.

A simple debugging routine

Symptom	First checks
A button does nothing	Check its id or data-action in index.html; check for the same name in an app.js listener or openDialog configuration.
A section is blank	Check that its HTML id exists and matches the selector in app.js. Check that the corresponding render function runs.
A page does not open	Compare data-page, page-section id, and navigate() page names.
A change is not visible	Save and refresh. For starter data, restore sample data because browser state overrides seed.
A phone layout overflows	Check CSS media queries and the width/min-width of the affected grid or table.
The grade looks unexpected	Check blank rows, totals, the points-based formula, and whether the course uses category weights.
A new task will not save	Check the required title field, form submit handler, config.save(), render(), and browser console for an error.

Small glossary

Term	Meaning in this app
Array	A list of records, written with square brackets.
Object	A record with named fields, written with curly braces.
DOM	The browser's live representation of the HTML page.
Selector	A CSS or JavaScript pattern used to find elements.
Event	A user action or browser signal such as click, change, or submit.
Render	Update page text and markup from the current data.
State	The current in-memory data for tasks, classes, budget, and grades.
localStorage	A simple key-value store in this browser profile.
Responsive design	Layout rules that adapt to screen width and user preferences.

A reliable edit loop: locate the section comment, change one value or rule, save, refresh, and check the feature that uses it. If the browser shows an error, read the first console error and the file/line it names.

APPENDIX A / Full index.html

Complete source listing for index.html. The line numbers identify the original source lines. Continuation rows are visual wraps for long lines.

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <meta name="theme-color" content="#17182F">
    <meta name="description" content="Plannex is your calm, everyday student planner for classes, tasks, spending, and grades.">
    <title>Plannex - your student planner</title>

    <!-- Fonts and the main visual style sheet. -->
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=Manrope:wght@400;500;600;700;800&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <div class="app-shell">
      <!-- Shared sidebar: the data-page names must match the page section IDs below. -->
      <aside class="sidebar" id="sidebar">
        <a class="brand" href="#overview" aria-label="Plannex home">
          <span class="brand-mark"><svg viewBox="300 110 420 420" aria-hidden="true"><image href="assets/plannex-logo.png" width="1110" height="835"></image></svg></span><span class="brand-name">PLANN<span class="brand-green">E</span>X<small>Make every day Count</small></span>
        </a>

        <p class="nav-label">Workspace</p>
        <nav class="primary-nav" aria-label="Main navigation">
          <a class="nav-link active" href="#overview" data-page="overview"><span class="nav-icon"> </span>Overview</a>
          <a class="nav-link" href="#planner" data-page="planner"><span class="nav-icon"> </span>Daily planner</a>
          <a class="nav-link" href="#classes" data-page="classes"><span class="nav-icon"> </span>Class schedule</a>
          <a class="nav-link" href="#budget" data-page="budget"><span class="nav-icon"> </span>My budget</a>
          <a class="nav-link" href="#grades" data-page="grades"><span class="nav-icon"> </span>Grade calculator</a>
        </nav>

        <div class="sidebar-bottom">
          <div class="sidebar-note">
            <span class="note-spark"> </span>
            <p>One thing at a time.<br><strong>You’ve got this.</strong></p>
            <span class="note-doodle"> </span>
          </div>
          <div class="profile">
            <div class="avatar">JS</div>
            <div><strong>Jamie Santos</strong><small>Student workspace</small></div>
            <span class="profile-dots">···</span>
          </div>
        </div>
      </aside>

      <main class="main-area">
        <!-- Shared header: JavaScript updates the page name and today's date. -->
        <header class="topbar">
          <button class="mobile-menu" id="menu-toggle" aria-label="Open menu"> </button>
          <div class="breadcrumb">
            <span>My workspace</span>
            <b>/</b>
            <strong id="crumb-current">Overview</strong>
          </div>
          <div class="topbar-right">
            <span class="today-chip"><span class="live-dot"></span><span id="today-label"></span></span>
            <button class="icon-button" id="reset-data" title="Restore sample data" aria-label="Restore sample data"> </button>
            <div class="avatar avatar-small">JS</div>
          </div>
        </header>

        <div class="content-wrap">
          <!-- Overview: a quick summary of tasks, classes, budget, and study momentum. -->
          <section class="page-section active-page" id="page-overview">
            <div class="welcome-row">
              <div>
                <p class="eyebrow" id="greeting">YOUR DAY, AT A GLANCE</p>
                <h1>A little more <span>organized.</span></h1>
                <p class="page-subtitle">A calm place to keep your student life together.</p>
              </div>
              <button class="button button-dark" data-action="add-task"><span> </span> Add a task</button>
            </div>

            <div class="stats-grid">
              <article class="stat-card">
                <div class="stat-top"><span>Tasks for today</span><span class="stat-icon icon-green"> </span></div>
                <div class="stat-number" id="stat-tasks">0 <small>tasks left</small></div>
                <div class="progress-track"><span id="task-progress"></span></div>
                <div class="stat-foot" id="task-foot">Keep your momentum going</div>
              </article>
              <article class="stat-card">
                <div class="stat-top"><span>Budget remaining</span><span class="stat-icon icon-peach"> </span></div>
                <div class="stat-number" id="stat-budget">₱0</div>
                <div class="stat-foot"><span class="mini-dot"></span><span id="budget-foot">of your monthly allowance</span></div>
              </article>
              <article class="stat-card">
                <div class="stat-top"><span>Current raw grade</span><span class="stat-icon icon-lilac"> </span></div>
                <div class="stat-number" id="stat-grade">- <small>%</small></div>
                <div class="stat-foot">From your entered score totals</div>
              </article>
            </div>

            <div class="dashboard-grid">
              <article class="panel today-panel">
                <div class="panel-heading">
```

```
                    <div><p class="eyebrow">MAKE IT COUNT</p><h2>Today’s plan <span class="heading-count" id="today-count">0</span></h2></div>
                    <a href="#planner" class="text-link">See all <span>→</span></a>
                  </div>
                  <div class="task-list" id="overview-tasks"></div>
                  <button class="add-inline" data-action="add-task">  Add a task</button>
                </article>

                <article class="panel schedule-panel">
                  <div class="panel-heading">
                    <div><p class="eyebrow">UP NEXT</p><h2>Your schedule</h2></div>
                    <a href="#classes" class="text-link">Full week <span>→</span></a>
                  </div>
                  <div class="next-class" id="next-class"></div>
                </article>

                <article class="panel spending-panel">
                  <div class="panel-heading">
                    <div><p class="eyebrow">MONEY CHECK-IN</p><h2>This month</h2></div>
                    <a href="#budget" class="text-link">Budget <span>→</span></a>
                  </div>
                  <div class="spend-total"><strong id="overview-spent">₱0</strong><span>spent so far</span></div>
                  <div class="budget-bar"><span id="budget-bar"></span></div>
                  <div class="budget-labels"><span id="spent-label">₱0 spent</span><span id="limit-label">₱0 limit</span></div>
                  <div class="recent-transactions" id="recent-transactions"></div>
                </article>

                <article class="panel focus-panel">
                  <div class="focus-copy">
                    <span class="focus-badge">A SMALL REMINDER</span>
                    <h2>Progress is still progress, <em>even on the busy days.</em></h2>
                    <p>Pick one thing to start. The rest can follow.</p>
                  </div>
                  <div class="focus-art"><span class="art-sun"></span><span class="art-star"> </span><span class="art-plant"> </span></div>
                </article>
              </div>
            </section>

            <!-- Daily planner: filter tasks by all, still to do, or completed. -->
            <section class="page-section" id="page-planner">
              <div class="welcome-row">
                <div>
                  <p class="eyebrow">YOUR EVERYDAY RESET</p>
                  <h1>Daily <span>planner.</span></h1>
                  <p class="page-subtitle">A simple list is a good place to begin.</p>
                </div>
                <button class="button button-dark" data-action="add-task">  Add a task</button>
              </div>

              <div class="planner-layout">
                <article class="panel planner-main">
                  <div class="panel-heading">
                    <div><p class="eyebrow" id="planner-date">TODAY</p><h2>My to-do list</h2></div>
                    <span class="heading-count" id="planner-count">0</span>
                  </div>
                  <div class="filter-tabs">
                    <button class="filter-tab selected" data-filter="all">All</button>
                    <button class="filter-tab" data-filter="open">To do</button>
                    <button class="filter-tab" data-filter="done">Completed</button>
                  </div>
                  <div class="task-list full-task-list" id="planner-tasks"></div>
                </article>

                <aside class="panel planner-side">
                  <p class="eyebrow">A GOOD RHYTHM</p>
                  <h2>Make space for breaks.</h2>
                  <p class="page-subtitle">Try a short focus session, then step away for a few minutes. Small resets add up.</p>
                  <div class="break-card"><span> </span><div><strong>Focus for 25 minutes</strong><small>One task. No rush.</small></div></div>
                  <div class="week-mini" id="week-mini"></div>
                </aside>
              </div>
            </section>

            <!-- Class schedule: JavaScript builds the weekly grid from the class data. -->
            <section class="page-section" id="page-classes">
              <div class="welcome-row">
                <div>
                  <p class="eyebrow">YOUR WEEK, IN ONE PLACE</p>
                  <h1>Class <span>schedule.</span></h1>
                  <p class="page-subtitle">Know where you need to be, and when.</p>
                </div>
                <button class="button button-dark" data-action="add-class">  Add a class</button>
              </div>

              <article class="panel timetable-panel">
                <div class="panel-heading">
                  <div><p class="eyebrow">SEMESTER 1 · 2026</p><h2>Weekly timetable</h2></div>
                  <div class="schedule-legend"><span><i class="legend-dot dot-green"></i>Classes</span></div>
                </div>
                <div class="table-scroll"><div class="timetable" id="timetable"></div></div>
              </article>
              <p class="section-hint">Tip: classes repeat each week. Add your own subjects to make this schedule yours.</p>
            </section>

            <!-- Budget: allowance and expenses are stored together in the planner state. -->
            <section class="page-section" id="page-budget">
              <div class="welcome-row">
                <div>
                  <p class="eyebrow">SPEND WITH A LITTLE MORE CLARITY</p>
                  <h1>My <span>budget.</span></h1>
                  <p class="page-subtitle">Keep track of what comes in and where it goes.</p>
                </div>
                <button class="button button-dark" data-action="add-expense">  Add expense</button>
              </div>

              <div class="budget-stats">
                <article class="stat-card budget-stat">
                  <span>Monthly allowance</span>
                  <strong id="budget-total">₱0</strong>
                  <button class="subtle-button" data-action="edit-budget">Edit allowance →</button>
```

```
              </article>
              <article class="stat-card budget-stat"><span>Total spent</span><strong id="budget-spent">₱0</strong><small>This month</small></arti
cle>
              <article class="stat-card budget-stat"><span>Left to spend</span><strong id="budget-remaining">₱0</strong><small id="budget-remaini
ng-note">You’re doing great</small></article>
            </div>

            <article class="panel expenses-panel">
              <div class="panel-heading">
                <div><p class="eyebrow">THE LITTLE THINGS ADD UP</p><h2>Recent expenses</h2></div>
                <select id="expense-filter" class="select-small" aria-label="Filter expenses">
                  <option value="all">All categories</option>
                  <option>Food</option>
                  <option>Transport</option>
                  <option>School</option>
                  <option>Personal</option>
                  <option>Other</option>
                </select>
              </div>
              <div id="expense-list" class="expense-list"></div>
            </article>
          </section>

          <!-- Grade calculator: enter assessment points, then compute the raw grade. -->
          <section class="page-section" id="page-grades">
            <div class="welcome-row">
              <div>
                <p class="eyebrow">KEEP AN EYE ON YOUR PROGRESS</p>
                <h1>Grade <span>calculator.</span></h1>
                <p class="page-subtitle">Enter the score and total points for each assessment.</p>
              </div>
            </div>

            <div class="grade-summary">
              <div>
                <span class="eyebrow">OVERALL RAW GRADE</span>
                <div class="grade-big" id="grade-average">-<small>%</small></div>
                <p class="grade-caption" id="grade-caption">Enter scores and totals, then compute your grade.</p>
              </div>
              <div class="grade-summary-art"><span> </span></div>
            </div>

            <article class="panel grades-panel">
              <div class="panel-heading">
                <div><p class="eyebrow">ASSESSMENT ITEMS</p><h2>Grade tracker</h2></div>
                <span class="grade-count" id="grade-count">0 rows</span>
              </div>
              <div class="grades-table-wrap">
                <table class="grades-table">
                  <thead>
                    <tr><th>CATEGORY</th><th>SCORE</th><th>TOTAL</th><th>RAW GRADE</th><th></th></tr>
                  </thead>
                  <tbody id="grade-rows"></tbody>
                </table>
              </div>
              <div class="grade-actions">
                <button class="button button-quiet" id="add-grade-row" type="button">  Add row</button>
                <button class="button button-dark" id="compute-grade" type="button">Compute grade</button>
              </div>
              <p class="grade-note">Raw grade = points earned ÷ points possible × 100. All entered items contribute by their point totals.</p>
            </article>
          </section>

          <footer class="footer-note">Made for the little things that make student life feel easier. <span> </span></footer>
        </div>
      </main>
    </div>

    <!-- Shared feedback message and entry dialog used by task, class, and budget actions. -->
    <div class="toast" id="toast" role="status" aria-live="polite"></div>
    <dialog id="entry-dialog">
      <form id="entry-form">
        <div class="dialog-head">
          <div><p class="eyebrow" id="dialog-eyebrow">QUICK ADD</p><h2 id="dialog-title">Add a task</h2></div>
          <button class="close-dialog" type="button" data-close-dialog aria-label="Close">×</button>
        </div>
        <div id="dialog-fields"></div>
        <div class="dialog-actions">
          <button class="button button-quiet" type="button" data-close-dialog>Cancel</button>
          <button class="button button-dark" id="dialog-save" type="submit">Save</button>
        </div>
      </form>
    </dialog>

    <!-- Load behavior after all page controls are in place. -->
    <script src="app.js"></script>
  </body>
</html>
```

APPENDIX B / Full styles.css

Complete source listing for styles.css. The line numbers identify the original source lines. Continuation rows are visual wraps for long lines.

```
/* Brand palette and shared design tokens */

:root {
  --midnight: #17182F;
  --surface: #242642;
  --green: #8DDA7A;
  --cloud: #F5F7FA;
  --slate: #667085;
  --ink: #24263d;
  --muted: #898da0;
  --line: #eceef3;
  --white: #fff;
  --peach: #f8e3d1;
  --lilac: #e8e1fa;
  --mint: #e5f5df;
  --shadow: 0 8px 28px rgba(23,24,47,.045);
}

* {
  box-sizing: border-box;
}

html {
  scroll-behavior: smooth;
}

body {
  margin: 0;
  background: var(--cloud);
  color: var(--ink);
  font-family: 'DM Sans',sans-serif;
  font-size: 14px;
  -webkit-font-smoothing: antialiased;
}

button,input,select {
  font: inherit;
}

button,a {
  -webkit-tap-highlight-color: transparent;
}

button {
  cursor: pointer;
}

.app-shell {
  min-height: 100vh;
}

.sidebar {
  width: 244px;
  position: fixed;
  inset: 0 auto 0 0;
  background: var(--midnight);
  color: #fff;
  padding: 28px 18px 18px;
  display: flex;
  flex-direction: column;
  z-index: 20;
}

/* Brand lockup and fixed navigation shell */

.brand {
  display: flex;
  align-items: center;
  gap: 11px;
  color: #fff;
  text-decoration: none;
  font: 800 22px Manrope,sans-serif;
  letter-spacing: -1px;
  padding: 0 12px 38px;
}

.brand-mark {
  width: 40px;
  height: 40px;
  flex: none;
  overflow: hidden;
  border: 1px solid #343550;
  border-radius: 12px;
  background: var(--surface);
  box-shadow: 0 3px 11px #090a1b55;
}

.brand-mark svg {
  display: block;
  width: 100%;
  height: 100%;
}

.brand-name {
  font: 800 15px/1.1 Manrope,sans-serif;
  letter-spacing: 1.4px;
  color: #fff;
}
```

```
 99
100  .brand-green {
101    color: var(--green);
102  }
103
104  .brand-name small {
105    display: block;
106    margin-top: 5px;
107    font: 400 9px/1 'DM Sans',sans-serif;
108    letter-spacing: .15px;
109    color: #a4a7b7;
110  }
111
112  .nav-label,.eyebrow {
113    font-size: 10px;
114    letter-spacing: 1.5px;
115    font-weight: 700;
116    color: #a4a7b7;
117    margin: 0;
118  }
119
120  .nav-label {
121    padding: 0 14px 12px;
122    color: #797d94;
123  }
124
125  .primary-nav {
126    display: grid;
127    gap: 5px;
128  }
129
130  .nav-link {
131    height: 43px;
132    display: flex;
133    align-items: center;
134    gap: 13px;
135    border-radius: 9px;
136    color: #b9bbca;
137    text-decoration: none;
138    padding: 0 13px;
139    font-size: 13px;
140    font-weight: 500;
141    transition: .2s;
142  }
143
144  .nav-link:hover {
145    background: #22233d;
146    color: #fff;
147  }
148
149  .nav-link.active {
150    background: #2c2d48;
151    color: #fff;
152    font-weight: 600;
153  }
154
155  .nav-icon {
156    width: 17px;
157    text-align: center;
158    font-size: 17px;
159    color: #a2a4b7;
160  }
161
162  .nav-link.active .nav-icon {
163    color: var(--green);
164  }
165
166  .sidebar-bottom {
167    margin-top: auto;
168  }
169
170  .sidebar-note {
171    position: relative;
172    margin: 0 2px 20px;
173    padding: 17px 15px;
174    background: #22233d;
175    border: 1px solid #30314c;
176    border-radius: 12px;
177    overflow: hidden;
178  }
179
180  .sidebar-note p {
181    font-size: 12px;
182    line-height: 1.8;
183    color: #bcbfce;
184    margin: 0 0 0 18px;
185  }
186
187  .sidebar-note strong {
188    font-size: 13px;
189    color: #fff;
190  }
191
192  .note-spark {
193    position: absolute;
194    left: 13px;
195    top: 17px;
196    color: var(--green);
197  }
198
199  .note-doodle {
200    position: absolute;
201    right: 18px;
202    bottom: 14px;
203    font-size: 28px;
204    color: #666a86;
205  }
206
207  .profile {
```

```
    display: flex;
    align-items: center;
    gap: 10px;
    padding: 15px 7px 0;
    border-top: 1px solid #2b2d48;
  }

  .avatar {
    width: 33px;
    height: 33px;
    flex: none;
    border-radius: 50%;
    background: #e9e0d6;
    color: #655143;
    display: grid;
    place-items: center;
    font-size: 10px;
    font-weight: 700;
  }

  .profile strong,.profile small {
    display: block;
  }

  .profile strong {
    font-size: 11px;
    font-weight: 600;
  }

  .profile small {
    font-size: 10px;
    color: #9295aa;
    margin-top: 4px;
  }

  .profile-dots {
    margin-left: auto;
    color: #a3a5b4;
    font-size: 20px;
  }

  .main-area {
    margin-left: 244px;
    min-height: 100vh;
  }

  .topbar {
    height: 68px;
    border-bottom: 1px solid #e9eaf0;
    display: flex;
    align-items: center;
    justify-content: space-between;
    padding: 0 44px;
    background: rgba(245,247,250,.9);
  }

  .breadcrumb {
    display: flex;
    align-items: center;
    gap: 11px;
    color: #9b9eaa;
    font-size: 11px;
  }

  .breadcrumb b {
    font-weight: 400;
    color: #c5c7d0;
  }

  .breadcrumb strong {
    color: #4a4c5e;
    font-weight: 600;
  }

  .topbar-right {
    display: flex;
    align-items: center;
    gap: 16px;
  }

  .today-chip {
    font-size: 11px;
    color: #65687b;
    display: flex;
    gap: 8px;
    align-items: center;
  }

  .live-dot,.mini-dot {
    width: 7px;
    height: 7px;
    border-radius: 50%;
    background: #83ca72;
    display: inline-block;
  }

  .icon-button,.mobile-menu {
    background: none;
    border: 0;
    color: #8b8ea0;
    font-size: 19px;
  }

  .icon-button:hover {
    color: var(--midnight);
  }

  .avatar-small {
    width: 28px;
```

```
    height: 28px;
    font-size: 9px;
  }

  .mobile-menu {
    display: none;
  }

  .content-wrap {
    max-width: 1270px;
    padding: 37px 44px 20px;
    margin: auto;
  }

  .welcome-row {
    display: flex;
    align-items: flex-end;
    justify-content: space-between;
    margin-bottom: 27px;
  }

  .eyebrow {
    color: #9a9dab;
  }

  .welcome-row h1 {
    font: 700 29px/1.2 Manrope,sans-serif;
    letter-spacing: -1.1px;
    margin: 9px 0 7px;
    color: var(--midnight);
  }

  .welcome-row h1 span {
    color: #7dbb6c;
  }

  .page-subtitle {
    color: #8b8e9c;
    margin: 0;
    font-size: 12px;
    line-height: 1.7;
  }

  .button {
    height: 39px;
    border: 0;
    border-radius: 8px;
    padding: 0 15px;
    font-size: 12px;
    font-weight: 600;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    gap: 8px;
    transition: .2s;
  }

  .button-dark {
    background: var(--midnight);
    color: #fff;
  }

  .button-dark:hover {
    background: #30314e;
    transform: translateY(-1px);
  }

  .button-dark span {
    color: var(--green);
    font-size: 17px;
  }

  .button-quiet {
    background: #f2f3f7;
    color: #55586c;
  }

  .page-section {
    display: none;
    animation: appear .25s ease;
  }

  .page-section.active-page {
    display: block;
  }

  @keyframes appear {
    from {
      opacity: .4;
      transform: translateY(4px);
    }

    to {
      opacity: 1;
      transform: none;
    }

  }

  /* Dashboard summary cards */

  .stats-grid {
    display: grid;
    grid-template-columns: repeat(3,1fr);
    gap: 15px;
    margin-bottom: 18px;
  }

  .stat-card,.panel {
```

```
  background: var(--white);
  border: 1px solid #eceef3;
  border-radius: 12px;
  box-shadow: var(--shadow);
}

.stat-card {
  padding: 17px 19px;
}

.stat-top {
  display: flex;
  justify-content: space-between;
  align-items: center;
  color: #797c8c;
  font-size: 11px;
}

.stat-icon {
  width: 29px;
  height: 29px;
  display: grid;
  place-items: center;
  border-radius: 8px;
  font-size: 15px;
}

.icon-green {
  background: #eaf7e5;
  color: #73b765;
}

.icon-peach {
  background: #fff0e5;
  color: #d68f63;
}

.icon-lilac {
  background: #f0eafb;
  color: #987cc8;
}

.stat-number {
  font: 700 25px Manrope,sans-serif;
  letter-spacing: -.8px;
  color: var(--midnight);
  margin: 8px 0 7px;
}

.stat-number small {
  font: 500 11px 'DM Sans',sans-serif;
  letter-spacing: 0;
  color: #9396a3;
}

.progress-track {
  height: 4px;
  background: #edf0ea;
  border-radius: 4px;
  overflow: hidden;
  margin: 11px 0 8px;
}

.progress-track span,.budget-bar span {
  height: 100%;
  display: block;
  background: var(--green);
  width: 0;
  transition: width .3s;
  border-radius: 4px;
}

.stat-foot {
  font-size: 10px;
  color: #a0a2ad;
  margin-top: 8px;
  display: flex;
  align-items: center;
  gap: 6px;
}

.dashboard-grid {
  display: grid;
  grid-template-columns: 1.1fr .9fr;
  gap: 16px;
}

.panel {
  padding: 19px;
}

.panel-heading {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 17px;
}

.panel-heading h2,.planner-side h2 {
  font: 700 16px Manrope,sans-serif;
  letter-spacing: -.35px;
  color: var(--midnight);
  margin: 6px 0 0;
}

.panel-heading .eyebrow {
  font-size: 9px;
}

```

```
.heading-count {
  display: inline-grid;
  place-items: center;
  background: #f0f1f5;
  border-radius: 20px;
  min-width: 21px;
  height: 21px;
  padding: 0 7px;
  font-size: 10px;
  color: #787b89;
  margin-left: 4px;
  vertical-align: 2px;
}

.text-link {
  font-size: 10px;
  color: #878a99;
  text-decoration: none;
}

.text-link span {
  color: #7ebc6e;
  margin-left: 4px;
  font-size: 14px;
}

.text-link:hover {
  color: var(--midnight);
}

/* Reusable task rows and schedule cards */

.task-list {
  display: grid;
}

.task-row {
  display: flex;
  align-items: center;
  gap: 11px;
  min-height: 49px;
  border-bottom: 1px solid #f0f1f5;
  padding: 7px 0;
}

.task-row:last-child {
  border-bottom: 0;
}

.check-button {
  width: 17px;
  height: 17px;
  flex: none;
  border-radius: 5px;
  border: 1.4px solid #d3d6df;
  background: #fff;
  color: var(--midnight);
  display: grid;
  place-items: center;
  font-size: 11px;
  padding: 0;
}

.check-button.checked {
  background: var(--green);
  border-color: var(--green);
}

.task-copy {
  min-width: 0;
  flex: 1;
}

.task-copy strong {
  display: block;
  font-size: 11px;
  font-weight: 600;
  color: #4d5061;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}

.task-copy small {
  display: block;
  font-size: 9px;
  color: #a4a6b1;
  margin-top: 4px;
}

.task-done .task-copy strong {
  text-decoration: line-through;
  color: #aaacb6;
}

.priority {
  font-size: 9px;
  padding: 4px 7px;
  border-radius: 12px;
  white-space: nowrap;
}

.priority.high {
  background: #fff0ec;
  color: #d77b64;
}

.priority.medium {
  background: #fff6e7;
```

```
  color: #c79c56;
}

.priority.low {
  background: #eff6ec;
  color: #76a56c;
}

.task-actions {
  opacity: 0;
  display: flex;
  gap: 2px;
}

.task-row:hover .task-actions,.task-row:focus-within .task-actions {
  opacity: 1;
}

.tiny-action {
  border: 0;
  background: none;
  color: #a0a2ac;
  padding: 3px;
  font-size: 13px;
}

.tiny-action:hover {
  color: #b65b55;
}

.add-inline {
  margin-top: 11px;
  border: 0;
  background: none;
  padding: 6px 0;
  color: #8d909d;
  font-size: 10px;
}

.add-inline:hover {
  color: var(--midnight);
}

.next-class {
  display: grid;
  gap: 11px;
}

.class-card {
  display: flex;
  align-items: center;
  gap: 12px;
  padding: 11px;
  background: #f8f9fb;
  border-radius: 8px;
}

.class-time {
  width: 53px;
  font-size: 10px;
  color: #787b8b;
  font-weight: 600;
}

.class-stripe {
  width: 3px;
  height: 34px;
  border-radius: 3px;
  background: #90d67c;
}

.class-info {
  flex: 1;
}

.class-info strong {
  display: block;
  font-size: 11px;
  color: #46495b;
}

.class-info small {
  display: block;
  margin-top: 4px;
  font-size: 9px;
  color: #9699a7;
}

.class-room {
  color: #999ba8;
  font-size: 9px;
}

.no-class {
  padding: 20px 0;
  color: #9295a3;
  font-size: 11px;
}

.spend-total {
  display: flex;
  align-items: baseline;
  gap: 8px;
}

.spend-total strong {
  font: 700 24px Manrope,sans-serif;
  color: var(--midnight);
}
```

```

.spend-total span {
  font-size: 10px;
  color: #999ba7;
}

.budget-bar {
  height: 7px;
  background: #eff1f0;
  border-radius: 6px;
  overflow: hidden;
  margin: 16px 0 7px;
}

.budget-bar span {
  background: #91d87d;
}

.budget-labels {
  display: flex;
  justify-content: space-between;
  color: #a0a2ac;
  font-size: 9px;
}

.recent-transactions {
  margin-top: 14px;
}

.transaction-row {
  display: flex;
  align-items: center;
  gap: 9px;
  padding: 9px 0;
  border-top: 1px solid #f0f1f5;
}

.transaction-icon {
  width: 27px;
  height: 27px;
  display: grid;
  place-items: center;
  background: #f3f4f7;
  border-radius: 8px;
  color: #828596;
}

.transaction-info {
  flex: 1;
}

.transaction-info strong {
  font-size: 10px;
  font-weight: 600;
  color: #585b6a;
  display: block;
}

.transaction-info small {
  font-size: 9px;
  color: #a0a2ad;
  display: block;
  margin-top: 3px;
}

.transaction-amount {
  font-size: 10px;
  font-weight: 600;
  color: #55586b;
}

.focus-panel {
  background: var(--midnight);
  color: #fff;
  position: relative;
  overflow: hidden;
  min-height: 177px;
  display: flex;
  align-items: center;
  padding-left: 23px;
}

.focus-copy {
  position: relative;
  z-index: 1;
  max-width: 75%;
}

.focus-badge {
  font-size: 8px;
  letter-spacing: 1.2px;
  font-weight: 700;
  color: var(--green);
}

.focus-copy h2 {
  font: 600 16px/1.5 Manrope,sans-serif;
  letter-spacing: -.3px;
  margin: 11px 0 7px;
  color: #fff;
}

.focus-copy h2 em {
  font-style: normal;
  color: var(--green);
}

.focus-copy p {
  font-size: 10px;
```

```
  color: #a3a6ba;
  margin: 0;
}

.focus-art {
  position: absolute;
  right: -10px;
  bottom: -27px;
  width: 148px;
  height: 148px;
  border-radius: 50%;
  background: #242642;
}

.art-sun {
  position: absolute;
  top: 19px;
  right: 22px;
  width: 41px;
  height: 41px;
  border-radius: 50%;
  background: var(--green);
}

.art-star {
  position: absolute;
  color: #d7b8fc;
  font-size: 21px;
  left: 26px;
  top: 20px;
}

.art-plant {
  position: absolute;
  bottom: 7px;
  right: 59px;
  color: #a3aa84;
  font-size: 65px;
  transform: rotate(-20deg);
}

.footer-note {
  padding: 28px 0 6px;
  text-align: center;
  color: #a4a6b1;
  font-size: 9px;
}

.footer-note span {
  color: #85c875;
  margin-left: 4px;
}

/* Planner, timetable, budget, and grade page layouts */

.planner-layout {
  display: grid;
  grid-template-columns: 1.55fr .75fr;
  gap: 16px;
}

.planner-main {
  padding: 22px;
}

.planner-side {
  padding: 22px;
  background: #fff;
}

.filter-tabs {
  display: flex;
  gap: 5px;
  border-bottom: 1px solid var(--line);
  margin-bottom: 8px;
}

.filter-tab {
  border: 0;
  background: none;
  color: #9699a6;
  padding: 9px 11px;
  font-size: 10px;
  border-bottom: 2px solid transparent;
}

.filter-tab.selected {
  color: var(--midnight);
  border-color: var(--green);
  font-weight: 600;
}

.full-task-list .task-row {
  min-height: 58px;
}

.planner-side h2 {
  font-size: 18px;
  margin: 12px 0 8px;
}

.planner-side .page-subtitle {
  font-size: 11px;
}

.break-card {
  display: flex;
  align-items: center;
  gap: 12px;
```

```
  background: #f5f8f1;
  border-radius: 9px;
  padding: 14px;
  margin-top: 22px;
}

.break-card>span {
  color: #7cbd6e;
  font-size: 20px;
}

.break-card strong,.break-card small {
  display: block;
}

.break-card strong {
  font-size: 11px;
  color: #535668;
}

.break-card small {
  font-size: 9px;
  color: #9b9eaa;
  margin-top: 4px;
}

.week-mini {
  margin-top: 24px;
  display: grid;
  grid-template-columns: repeat(7,1fr);
  gap: 4px;
}

.mini-day {
  text-align: center;
  font-size: 8px;
  color: #a4a6b2;
}

.mini-day b {
  display: grid;
  place-items: center;
  height: 26px;
  width: 26px;
  margin: 7px auto 0;
  border-radius: 50%;
  font-size: 9px;
  font-weight: 500;
  color: #707384;
}

.mini-day.current b {
  background: var(--green);
  color: var(--midnight);
  font-weight: 700;
}

.timetable-panel {
  padding: 22px;
}

.schedule-legend {
  font-size: 10px;
  color: #9295a3;
}

.schedule-legend span {
  display: flex;
  align-items: center;
  gap: 6px;
}

.legend-dot {
  width: 7px;
  height: 7px;
  border-radius: 50%;
  background: var(--green);
}

.table-scroll {
  overflow-x: auto;
}

.timetable {
  min-width: 680px;
  display: grid;
  grid-template-columns: 62px repeat(5,1fr);
  border: 1px solid #eff0f4;
  border-radius: 8px;
  overflow: hidden;
}

.time-head,.day-head {
  background: #fafbfc;
  padding: 13px 9px;
  font-size: 9px;
  font-weight: 700;
  letter-spacing: .5px;
  color: #8e91a0;
  text-align: center;
  border-bottom: 1px solid #eff0f4;
}

.time-head {
  border-right: 1px solid #eff0f4;
}

.time-slot {
  min-height: 66px;
```

```
  padding: 8px 6px;
  text-align: center;
  font-size: 9px;
  color: #a4a6b1;
  border-right: 1px solid #f0f1f5;
  border-bottom: 1px solid #f0f1f5;
}

.class-slot {
  min-height: 66px;
  padding: 5px;
  border-bottom: 1px solid #f0f1f5;
  border-right: 1px solid #f0f1f5;
  position: relative;
}

.class-block {
  height: 100%;
  min-height: 50px;
  border-radius: 6px;
  background: #edf8e9;
  border-left: 3px solid #8DDA7A;
  padding: 7px 7px;
  color: #4e6b48;
  cursor: pointer;
}

.class-block:hover {
  background: #e1f2db;
}

.class-block strong {
  font-size: 9px;
  display: block;
  line-height: 1.35;
}

.class-block small {
  font-size: 8px;
  display: block;
  color: #799474;
  margin-top: 4px;
}

.section-hint {
  font-size: 10px;
  color: #9b9daa;
  margin: 13px 3px;
}

.budget-stats {
  display: grid;
  grid-template-columns: repeat(3,1fr);
  gap: 14px;
  margin-bottom: 17px;
}

.budget-stat {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  gap: 10px;
}

.budget-stat>span {
  font-size: 10px;
  color: #9295a3;
}

.budget-stat>strong {
  font: 700 23px Manrope,sans-serif;
  letter-spacing: -.7px;
  color: var(--midnight);
}

.budget-stat small {
  font-size: 10px;
  color: #a2a4ae;
}

.subtle-button {
  border: 0;
  background: none;
  color: #7eae72;
  padding: 0;
  font-size: 10px;
}

.expenses-panel {
  padding: 21px;
}

.select-small {
  border: 1px solid #eceef3;
  border-radius: 7px;
  padding: 8px 10px;
  background: white;
  color: #777a89;
  font-size: 10px;
  outline: none;
}

.expense-row {
  display: grid;
  grid-template-columns: 35px 1fr auto auto;
  gap: 12px;
  align-items: center;
  border-bottom: 1px solid #f0f1f5;
  padding: 12px 1px;
```

```
}

.expense-row:last-child {
  border: 0;
}

.expense-icon {
  width: 31px;
  height: 31px;
  border-radius: 9px;
  background: #f2f3f6;
  display: grid;
  place-items: center;
  font-size: 14px;
}

.expense-copy strong,.expense-copy small {
  display: block;
}

.expense-copy strong {
  font-size: 11px;
  color: #545768;
}

.expense-copy small {
  font-size: 9px;
  color: #a1a3ae;
  margin-top: 4px;
}

.expense-category {
  font-size: 9px;
  color: #9093a0;
  background: #f7f8fa;
  border-radius: 12px;
  padding: 5px 8px;
}

.expense-amount {
  font-size: 11px;
  font-weight: 600;
  color: #4f5263;
  min-width: 60px;
  text-align: right;
}

.grade-summary {
  border-radius: 12px;
  background: var(--midnight);
  min-height: 150px;
  color: white;
  padding: 24px 29px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 17px;
  overflow: hidden;
  position: relative;
}

.grade-summary .eyebrow {
  color: #a3a5b8;
}

.grade-big {
  font: 700 39px Manrope,sans-serif;
  letter-spacing: -1.5px;
  color: var(--green);
  margin: 6px 0 2px;
}

.grade-big small {
  font-size: 15px;
  margin-left: 3px;
}

.grade-caption {
  margin: 0;
  font-size: 10px;
  color: #b0b2c2;
}

.grade-summary-art {
  width: 135px;
  height: 135px;
  background: #242642;
  border-radius: 50%;
  margin: -40px 20px -60px 0;
  display: grid;
  place-items: center;
  color: #b8a3e2;
  font-size: 47px;
}

.grades-panel {
  padding: 22px;
}

.grade-count {
  font-size: 10px;
  color: #999ba8;
}

.grades-table {
  border-collapse: collapse;
  width: 100%;
  min-width: 550px;
}
```

```
1298
1299  .grades-table th {
1300    text-align: left;
1301    padding: 10px 8px;
1302    border-bottom: 1px solid #eff0f4;
1303    color: #a2a4af;
1304    font-size: 8px;
1305    letter-spacing: 1px;
1306    font-weight: 700;
1307  }
1308
1309  .grades-table td {
1310    padding: 13px 8px;
1311    border-bottom: 1px solid #f0f1f5;
1312    color: #646778;
1313    font-size: 10px;
1314  }
1315
1316  .grades-table tr:last-child td {
1317    border-bottom: 0;
1318  }
1319
1320  .grades-table td:first-child {
1321    font-weight: 600;
1322    color: #4c4f60;
1323  }
1324
1325  .grades-table input {
1326    width: 76px;
1327    padding: 6px 7px;
1328    border: 1px solid #e8eaf0;
1329    border-radius: 6px;
1330    color: #4d5060;
1331    font-size: 10px;
1332    outline: none;
1333  }
1334
1335  .grades-table select {
1336    width: 100%;
1337    min-width: 112px;
1338    padding: 6px 7px;
1339    border: 1px solid #e8eaf0;
1340    border-radius: 6px;
1341    background: #fff;
1342    color: #4d5060;
1343    font-size: 10px;
1344    outline: none;
1345  }
1346
1347  .grades-table input:focus {
1348    border-color: #8DDA7A;
1349  }
1350
1351  .grades-table select:focus {
1352    border-color: #8DDA7A;
1353  }
1354
1355  .grade-raw {
1356    color: #4f5263!important;
1357    font-weight: 600;
1358    white-space: nowrap;
1359  }
1360
1361  .grade-actions {
1362    display: flex;
1363    justify-content: center;
1364    gap: 10px;
1365    margin: 18px 0 4px;
1366  }
1367
1368  .row-action {
1369    border: 0;
1370    background: transparent;
1371    color: #a2a4af;
1372    font-size: 13px;
1373  }
1374
1375  .row-action:hover {
1376    color: #cb7164;
1377  }
1378
1379  .grade-note {
1380    font-size: 9px;
1381    color: #a0a2ac;
1382    margin: 12px 7px 0;
1383  }
1384
1385  .empty-state {
1386    text-align: center;
1387    color: #a1a3ae;
1388    padding: 25px 10px;
1389    font-size: 11px;
1390  }
1391
1392  /* Entry dialog, form fields, and feedback toast */
1393
1394  .dialog-head {
1395    display: flex;
1396    align-items: flex-start;
1397    justify-content: space-between;
1398    margin-bottom: 20px;
1399  }
1400
1401  .dialog-head h2 {
1402    font: 700 20px Manrope,sans-serif;
1403    color: var(--midnight);
1404    margin: 7px 0 0;
1405    letter-spacing: -.5px;
1406  }
```

```
1407
1408  .close-dialog {
1409    border: 0;
1410    background: none;
1411    color: #9295a3;
1412    font-size: 22px;
1413    line-height: 1;
1414  }
1415
1416  .dialog-field {
1417    margin-bottom: 13px;
1418  }
1419
1420  .dialog-field label {
1421    display: block;
1422    color: #777a89;
1423    font-size: 10px;
1424    font-weight: 600;
1425    margin-bottom: 6px;
1426  }
1427
1428  .dialog-field input,.dialog-field select {
1429    width: 100%;
1430    height: 38px;
1431    border: 1px solid #e8eaf0;
1432    border-radius: 7px;
1433    padding: 0 10px;
1434    color: #4b4e5f;
1435    background: #fff;
1436    outline: none;
1437    font-size: 11px;
1438  }
1439
1440  .dialog-field input:focus,.dialog-field select:focus {
1441    border-color: #8DDA7A;
1442    box-shadow: 0 0 0 3px #8dda7a25;
1443  }
1444
1445  .dialog-grid {
1446    display: grid;
1447    grid-template-columns: 1fr 1fr;
1448    gap: 11px;
1449  }
1450
1451  .dialog-actions {
1452    display: flex;
1453    justify-content: flex-end;
1454    gap: 8px;
1455    margin-top: 20px;
1456  }
1457
1458  .dialog-actions .button {
1459    height: 36px;
1460  }
1461
1462  dialog {
1463    width: min(420px,calc(100% - 30px));
1464    border: 1px solid #eceef3;
1465    border-radius: 13px;
1466    padding: 23px 24px;
1467    box-shadow: 0 25px 80px #17182f35;
1468  }
1469
1470  dialog::backdrop {
1471    background: #17182f65;
1472    backdrop-filter: blur(2px);
1473  }
1474
1475  .toast {
1476    position: fixed;
1477    bottom: 23px;
1478    left: 50%;
1479    transform: translate(-50%,15px);
1480    padding: 10px 15px;
1481    border-radius: 8px;
1482    background: var(--midnight);
1483    color: white;
1484    font-size: 11px;
1485    opacity: 0;
1486    pointer-events: none;
1487    transition: .22s;
1488    z-index: 40;
1489    box-shadow: 0 6px 20px #17182f30;
1490  }
1491
1492  .toast.show {
1493    opacity: 1;
1494    transform: translate(-50%,0);
1495  }
1496
1497  /* Tablet adjustments */
1498
1499  @media(max-width:1050px) {
1500    .content-wrap {
1501      padding: 30px 27px 18px;
1502    }
1503
1504    .topbar {
1505      padding: 0 27px;
1506    }
1507
1508    .sidebar {
1509      width: 220px;
1510    }
1511
1512    .main-area {
1513      margin-left: 220px;
1514    }
1515
```

```
  .dashboard-grid {
    gap: 12px;
  }

  .focus-copy {
    max-width: 82%;
  }

}

@media(max-width:800px) {
  .sidebar {
    transform: translateX(-100%);
    transition: transform .25s;
    width: 245px;
    box-shadow: 10px 0 40px #17182f22;
  }

  .sidebar.open {
    transform: translateX(0);
  }

  .main-area {
    margin-left: 0;
  }

  .mobile-menu {
    display: block;
    padding: 4px 8px 4px 0;
  }

  .topbar {
    padding: 0 20px;
  }

  .breadcrumb {
    margin-right: auto;
  }

  .content-wrap {
    padding: 27px 20px 18px;
  }

  /* Tablet planner and timetable layout */

  .planner-layout {
    grid-template-columns: 1fr .72fr;
  }

  .dashboard-grid {
    grid-template-columns: 1fr 1fr;
  }

  .welcome-row h1 {
    font-size: 26px;
  }

}

/* Phone adjustments */

@media(max-width:600px) {
  .topbar {
    height: 59px;
    padding: 0 16px;
  }

  .topbar-right {
    gap: 10px;
  }

  .today-chip {
    font-size: 0;
  }

  .today-chip .live-dot {
    font-size: initial;
  }

  .content-wrap {
    padding: 24px 15px 15px;
  }

  .welcome-row {
    align-items: flex-start;
    gap: 12px;
    margin-bottom: 21px;
  }

  .welcome-row h1 {
    font-size: 23px;
  }

  .page-subtitle {
    font-size: 11px;
    max-width: 250px;
  }

  .welcome-row>.button {
    white-space: nowrap;
    padding: 0 10px;
    height: 36px;
    font-size: 10px;
    margin-top: 11px;
  }

  /* Compact dashboard summary cards */

  .stats-grid {
```

```
    grid-template-columns: 1fr;
    gap: 9px;
  }

  .stat-card {
    padding: 14px 16px;
  }

  .stat-number {
    font-size: 22px;
    margin: 5px 0;
  }

  .stat-icon {
    width: 27px;
    height: 27px;
  }

  .stat-foot {
    margin-top: 5px;
  }

  .progress-track {
    margin: 8px 0 5px;
  }

  .dashboard-grid {
    grid-template-columns: 1fr;
    gap: 10px;
  }

  .panel {
    padding: 16px;
  }

  .focus-panel {
    min-height: 160px;
  }

  /* Phone planner, budget, and grade layout */

  .planner-layout {
    grid-template-columns: 1fr;
  }

  .planner-side {
    order: -1;
  }

  .planner-side .week-mini {
    margin-top: 15px;
  }

  .budget-stats {
    grid-template-columns: 1fr;
    gap: 9px;
  }

  .budget-stat {
    display: grid;
    grid-template-columns: 1fr auto;
    gap: 7px 10px;
    align-items: center;
  }

  .budget-stat>span {
    grid-column: 1;
  }

  .budget-stat>strong {
    grid-column: 1;
    grid-row: 2;
  }

  .budget-stat .subtle-button,.budget-stat small {
    grid-column: 2;
    grid-row: 2;
  }

  .expenses-panel {
    padding: 15px;
  }

  .expense-row {
    grid-template-columns: 32px 1fr auto;
    gap: 8px;
  }

  .expense-category {
    display: none;
  }

  .expense-amount {
    min-width: 54px;
  }

  .grade-summary {
    padding: 20px;
    min-height: 135px;
  }

  .grade-summary-art {
    margin-right: -35px;
  }

  .grades-panel {
    padding: 16px;
  }

```

```
  .grades-table-wrap {
    overflow-x: auto;
  }

  .footer-note {
    line-height: 1.7;
  }

  .dialog-grid {
    grid-template-columns: 1fr 1fr;
  }

}

@media(prefers-reduced-motion:reduce) {
  *,*:before,*:after {
    scroll-behavior: auto!important;
    animation-duration: .01ms!important;
    transition-duration: .01ms!important;
  }

}
```

APPENDIX C / Full app.js

Complete source listing for app.js. The line numbers identify the original source lines. Continuation rows are visual wraps for long lines.

```
(() => {
  // 1. Starter data and settings
  // These values set the first-run sample content and the timetable/grade options.
  const KEY = "plannex-student-planner-v1";
  const DAYS = ["Mon", "Tue", "Wed", "Thu", "Fri"];
  const HOURS = [
    "8:00",
    "9:00",
    "10:00",
    "11:00",
    "12:00",
    "1:00",
    "2:00",
    "3:00",
    "4:00",
  ];
  const GRADE_CATEGORIES = ["Quiz", "Long Quiz", "Assignment", "Exam"];
  const seed = {
    allowance: 12000,
    tasks: [
      {
        id: "t1",
        title: "Finish biology lab notes",
        due: "Today · 11:30 AM",
        priority: "High",
        done: false,
      },
      {
        id: "t2",
        title: "Read chapter 4 for literature",
        due: "Today · 2:00 PM",
        priority: "Medium",
        done: false,
      },
      {
        id: "t3",
        title: "Submit statistics problem set",
        due: "Today · 5:00 PM",
        priority: "High",
        done: true,
      },
      {
        id: "t4",
        title: "Pack things for tomorrow",
        due: "Today · Anytime",
        priority: "Low",
        done: false,
      },
    ],
    classes: [
      {
        id: "c1",
        day: "Mon",
        time: "9:00",
        name: "Biology 101",
        room: "Science Hall · 204",
      },
      {
        id: "c2",
        day: "Mon",
        time: "11:00",
        name: "Literature & Society",
        room: "Arts Building · 12",
      },
      {
        id: "c3",
        day: "Mon",
        time: "2:00",
        name: "Statistics",
        room: "Main Building · 305",
      },
      {
        id: "c4",
        day: "Tue",
        time: "8:00",
        name: "Communication",
        room: "Arts Building · 18",
      },
      {
        id: "c5",
        day: "Tue",
        time: "10:00",
        name: "Biology 101",
        room: "Science Hall · 204",
      },
      {
        id: "c6",
        day: "Tue",
        time: "1:00",
        name: "Physical Education",
        room: "Gymnasium",
      },
      {
        id: "c7",
        day: "Wed",
        time: "9:00",
        name: "Biology 101",
        room: "Science Hall · 204",
```

```
      },
      {
        id: "c8",
        day: "Wed",
        time: "11:00",
        name: "Literature & Society",
        room: "Arts Building · 12",
      },
      {
        id: "c9",
        day: "Wed",
        time: "2:00",
        name: "Statistics",
        room: "Main Building · 305",
      },
      {
        id: "c10",
        day: "Thu",
        time: "8:00",
        name: "Communication",
        room: "Arts Building · 18",
      },
      {
        id: "c11",
        day: "Thu",
        time: "10:00",
        name: "Biology 101",
        room: "Science Hall · 204",
      },
      {
        id: "c12",
        day: "Fri",
        time: "9:00",
        name: "Literature & Society",
        room: "Arts Building · 12",
      },
      {
        id: "c13",
        day: "Fri",
        time: "11:00",
        name: "Statistics",
        room: "Main Building · 305",
      },
    ],
    expenses: [
      {
        id: "e1",
        name: "Lunch at campus",
        category: "Food",
        amount: 145,
        date: "2026-10-17",
      },
      {
        id: "e2",
        name: "Jeepney fare",
        category: "Transport",
        amount: 40,
        date: "2026-10-16",
      },
      {
        id: "e3",
        name: "Notebook & pens",
        category: "School",
        amount: 185,
        date: "2026-10-15",
      },
      {
        id: "e4",
        name: "Iced coffee",
        category: "Food",
        amount: 120,
        date: "2026-10-14",
      },
    ],
    gradesComputed: false,
    grades: [
      { id: "g1", category: "Quiz", score: 18, total: 20 },
      { id: "g2", category: "Quiz", score: 15, total: 20 },
      { id: "g3", category: "Long Quiz", score: 42, total: 50 },
      { id: "g4", category: "Assignment", score: 27, total: 30 },
      { id: "g5", category: "Exam", score: 84, total: 100 },
    ],
  };

  // 2. Saved state and small helpers
  // Short DOM helpers keep the render functions readable.
  const $ = (s, root = document) => root.querySelector(s);
  const $$ = (s, root = document) => [...root.querySelectorAll(s)];
  let state;
  try {
    state = JSON.parse(localStorage.getItem(KEY)) || structuredClone(seed);
    // Keep old subject grades in storage before switching to assessment rows.
    if (state.grades?.some((grade) => !("category" in grade))) {
      state.previousSubjectGrades = structuredClone(state.grades);
      state.grades = structuredClone(seed.grades);
      state.gradesComputed = false;
    }
    if (typeof state.gradesComputed !== "boolean") state.gradesComputed = false;
  } catch {
    state = structuredClone(seed);
  }
  // Save the complete planner snapshot after each change so the sections stay in sync.
  const persist = () => localStorage.setItem(KEY, JSON.stringify(state));
  const money = (value) =>
    new Intl.NumberFormat("en-PH", {
      style: "currency",
      currency: "PHP",
      maximumFractionDigits: 0,
    }).format(Number(value) || 0);
```

```
  const EXPENSE_ICONS = {
    Food: " ",
    Transport: " ",
    School: " ",
    Personal: " ",
    Other: "·",
  };
  const escapeHtml = (value) =>
    String(value).replace(
      /[&<>"']/g,
      (c) =>
        ({
          "&": "&amp;",
          "<": "&lt;",
          ">": "&gt;",
          '"': "&quot;",
          "'": "&#39;",
        })[c],
    );
  const makeId = () =>
    `${Date.now().toString(36)}${Math.random().toString(36).slice(2, 7)}`;
  const today = new Date();
  const friendlyDate = today.toLocaleDateString("en-PH", {
    weekday: "long",
    month: "long",
    day: "numeric",
  });
  $("#today-label").textContent = friendlyDate;
  $("#planner-date").textContent = `TODAY · ${friendlyDate.toUpperCase()}`;
  $("#greeting").textContent =
    `${today.getHours() < 12 ? "GOOD MORNING" : today.getHours() < 18 ? "GOOD AFTERNOON" : "GOOD EVENING"}, JAMIE`;
  const toast = (message) => {
    const node = $("#toast");
    node.textContent = message;
    node.classList.add("show");
    clearTimeout(toast.timer);
    toast.timer = setTimeout(() => node.classList.remove("show"), 2200);
  };

  // 3. Rendering helpers
  // These functions turn saved state into the cards, tables, and totals on screen.
  // Shared markup keeps task rows consistent on the dashboard and full planner page.
  function tasksMarkup(tasks, controls = true) {
    if (!tasks.length)
      return '<div class="empty-state">Nothing here just yet. Add a task when you're ready.</div>';
    return tasks
      .map(
        (task) =>
          `<div class="task-row ${task.done ? "task-done" : ""}"><button class="check-button ${task.done ? "checked" : ""}" data-toggle-task="${task.id}" aria-label="${task.done ? "Mark incomplete" : "Mark complete"}">${task.done ? " " : ""}</button><div class="task-copy"><strong>${escapeHtml(task.title)}</strong><small>${escapeHtml(task.due || "Today")}</small></div><span class="priority ${(task.priority || "Medium").toLowerCase()}">${escapeHtml(task.priority || "Medium")}</span>${controls ? `<span class="task-actions"><button class="tiny-action" data-delete-task="${task.id}" aria-label="Delete task">×</button></span>` : ""}</div>`,
      )
      .join("");
  }
  function getRowGrade(row) {
    if (row.score === "" || row.score == null || row.total === "" || row.total == null)
      return null;
    const score = Number(row.score);
    const total = Number(row.total);
    return Number.isFinite(score) && Number.isFinite(total) && total > 0
      ? (score / total) * 100
      : null;
  }
  // Calculate a raw grade from points, so items with larger totals naturally count more.
  function getOverallRawGrade() {
    const valid = state.grades.filter((row) => getRowGrade(row) !== null);
    if (!valid.length) return null;
    const earned = valid.reduce((sum, row) => sum + Number(row.score), 0);
    const possible = valid.reduce((sum, row) => sum + Number(row.total), 0);
    return possible > 0 ? (earned / possible) * 100 : null;
  }
  function renderTasks() {
    const remaining = state.tasks.filter((t) => !t.done).length,
      done = state.tasks.length - remaining;
    $("#stat-tasks").innerHTML =
      `${remaining} <small>${remaining === 1 ? "task" : "tasks"} left</small>`;
    $("#today-count").textContent = state.tasks.length;
    $("#planner-count").textContent = remaining;
    $("#task-foot").textContent = state.tasks.length
      ? `${done} of ${state.tasks.length} completed`
      : "Add a task to get started";
    $("#task-progress").style.width =
      `${state.tasks.length ? (done / state.tasks.length) * 100 : 0}%`;
    $("#overview-tasks").innerHTML = tasksMarkup(state.tasks.slice(0, 4));
    const filter = $(".filter-tab.selected")?.dataset.filter || "all";
    $("#planner-tasks").innerHTML = tasksMarkup(
      state.tasks.filter(
        (t) => filter === "all" || (filter === "open" ? !t.done : t.done),
      ),
    );
  }
  function renderBudget() {
    const spent = state.expenses.reduce((sum, e) => sum + Number(e.amount), 0),
      remaining = Number(state.allowance) - spent,
      ratio =
        Number(state.allowance) > 0
          ? Math.min(100, (spent / Number(state.allowance)) * 100)
          : 100;
    $("#stat-budget").textContent = money(remaining);
    $("#budget-foot").textContent =
      `of your ${money(state.allowance)} monthly allowance`;
    $("#overview-spent").textContent = money(spent);
    $("#spent-label").textContent = `${money(spent)} spent`;
    $("#limit-label").textContent = `${money(state.allowance)} limit`;
    $("#budget-bar").style.width = `${ratio}%`;
    $("#budget-total").textContent = money(state.allowance);
    $("#budget-spent").textContent = money(spent);
```

```
    $("#budget-remaining").textContent = money(remaining);
    $("#budget-remaining").style.color = remaining < 0 ? "#ce8065" : "";
    $("#budget-remaining-note").textContent =
      remaining < 0 ? "Over your monthly allowance" : "You're doing great";
    const icons = EXPENSE_ICONS;
    $("#recent-transactions").innerHTML =
      state.expenses
        .slice(0, 2)
        .map(
          (e) =>
            `<div class="transaction-row"><span class="transaction-icon">${icons[e.category] || "·"}</span><span class="transaction-info"><strong>${escapeHtml(e.name)}</strong><small>${escapeHtml(e.category)}</small></span><span class="transaction-amount">-${money(e.amount)}</span></div>`,
        )
        .join("") || '<div class="empty-state">No expenses yet.</div>';
    renderExpenses();
  }
  function renderExpenses() {
    const filter = $("#expense-filter").value;
    const rows = state.expenses.filter(
      (e) => filter === "all" || e.category === filter,
    );
    const icons = EXPENSE_ICONS;
    $("#expense-list").innerHTML = rows.length
      ? rows
          .map(
            (e) =>
              `<div class="expense-row"><span class="expense-icon">${icons[e.category] || "·"}</span><span class="expense-copy"><strong>${escapeHtml(e.name)}</strong><small>${new Date(`${e.date}T12:00:00`).toLocaleDateString("en-PH", { month: "short", day: "numeric", year: "numeric" })}</small></span><span class="expense-category">${escapeHtml(e.category)}</span><span class="expense-amount">-${money(e.amount)}</span></div>`,
          )
          .join("")
      : '<div class="empty-state">No expenses in this category.</div>';
  }
  function renderGradeSummary() {
    const avg = state.gradesComputed ? getOverallRawGrade() : null;
    $("#stat-grade").innerHTML =
      avg === null
        ? "- <small>%</small>"
        : `${avg.toFixed(1)}<small>%</small>`;
    $("#grade-average").innerHTML =
      avg === null ? "-<small>%</small>" : `${avg.toFixed(1)}<small>%</small>`;
    $("#grade-caption").textContent =
      avg === null
        ? "Enter scores and totals, then compute your grade."
        : avg >= 90
          ? "Excellent work - your effort is showing."
          : avg >= 75
            ? "You're making steady progress. Keep going."
            : "Use this result to guide your next study session.";
    $("#grade-count").textContent =
      `${state.grades.length} ${state.grades.length === 1 ? "row" : "rows"}`;
  }
  function renderGrades() {
    renderGradeSummary();
    $("#grade-rows").innerHTML = state.grades.length
      ? state.grades
          .map(
            (row) => {
              const rawGrade = getRowGrade(row);
              return `<tr><td><select data-grade-row="${row.id}" data-grade-field="category" aria-label="Assessment category">${GRADE_CATEGORIES.map((category) => `<option value="${category}" ${row.category === category ? "selected" : ""}>${category}</option>`).join("")}</select></td><td><input type="number" min="0" step="any" value="${row.score ?? ""}" placeholder="Score" data-grade-row="${row.id}" data-grade-field="score" aria-label="Score"></td><td><input type="number" min="0.01" step="any" value="${row.total ?? ""}" placeholder="Total" data-grade-row="${row.id}" data-grade-field="total" aria-label="Total possible points"></td><td class="grade-raw">${state.gradesComputed && rawGrade !== null ? `${rawGrade.toFixed(1)}%` : "-"}</td><td><button class="row-action" data-delete-grade="${row.id}" aria-label="Remove grade row">×</button></td></tr>`;
            },
          )
          .join("")
      : '<tr><td colspan="5"><div class="empty-state">No grade rows yet. Add a row to get started.</div></td></tr>';
  }
  // The timetable uses 1:00-4:00 for afternoon classes; normalize those for comparisons.
  function toMinutes(time) {
    const [hour, minute] = time.split(":").map(Number);
    const normalized = hour >= 1 && hour <= 4 ? hour + 12 : hour;
    return normalized * 60 + (minute || 0);
  }
  function renderClasses() {
    const grid = $("#timetable");
    grid.innerHTML = `<div class="time-head">TIME</div>${DAYS.map((d) => `<div class="day-head">${d.toUpperCase()}</div>`).join("")}`;
    for (const hour of HOURS) {
      grid.insertAdjacentHTML(
        "beforeend",
        `<div class="time-slot">${hour}</div>`,
      );
      for (const day of DAYS) {
        const item = state.classes.find(
          (c) => c.day === day && parseInt(c.time, 10) === parseInt(hour, 10),
        );
        grid.insertAdjacentHTML(
          "beforeend",
          `<div class="class-slot">${item ? `<div class="class-block" title="Click to remove this class" data-delete-class="${item.id}"><strong>${escapeHtml(item.name)}</strong><small>${escapeHtml(item.room)}</small></div>` : ""}</div>`,
        );
      }
    }
    const todayCode = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"][
      today.getDay()
    ];
    const now = today.getHours() * 60 + today.getMinutes();
    const upcoming = state.classes
      .filter((c) => c.day === todayCode && toMinutes(c.time) >= now)
      .sort((a, b) => toMinutes(a.time) - toMinutes(b.time))[0];
    const next = $("#next-class");
    next.innerHTML = upcoming
      ? `<div class="class-card"><span class="class-time">${escapeHtml(upcoming.time)}</span><span class="class-stripe"></span><span class="class-info"><strong>${escapeHtml(upcoming.name)}</strong><small>${escapeHtml(upcoming.room)}</small></span></div><div class="no-class">Your next class today.</div>`
      : `<div class="no-class">${DAYS.includes(todayCode) ? "No more classes today. Take a breather." : "No classes scheduled today. Enjoy your day."}</div>`;
```

```
  }
  function renderWeek() {
    const start = new Date(today);
    start.setDate(today.getDate() - ((today.getDay() + 6) % 7));
    $("#week-mini").innerHTML = Array.from({ length: 7 }, (_, i) => {
      const date = new Date(start);
      date.setDate(start.getDate() + i);
      return `<div class="mini-day ${i === (today.getDay() + 6) % 7 ? "current" : ""}">${["M", "T", "W", "T", "F", "S", "S"][i]}<b>${date.getDate()}</b></div>`;
    }).join("");
  }
  function render() {
    renderTasks();
    renderBudget();
    renderGrades();
    renderClasses();
    renderWeek();
    persist();
  }

  // 4. Navigation and page interactions
  function navigate(page) {
    if (!$("#page-" + page)) page = "overview";
    $$(".page-section").forEach((n) =>
      n.classList.toggle("active-page", n.id === `page-${page}`),
    );
    $$(".nav-link").forEach((n) =>
      n.classList.toggle("active", n.dataset.page === page),
    );
    $("#crumb-current").textContent = {
      overview: "Overview",
      planner: "Daily planner",
      classes: "Class schedule",
      budget: "My budget",
      grades: "Grade calculator",
    }[page];
    history.replaceState(null, "", `#${page}`);
    $("#sidebar").classList.remove("open");
  }
  $$(".nav-link").forEach((link) =>
    link.addEventListener("click", (e) => {
      e.preventDefault();
      navigate(link.dataset.page);
    }),
  );
  $("#menu-toggle").addEventListener("click", () =>
    $("#sidebar").classList.toggle("open"),
  );
  // Delegate clicks because task, class, and grade controls are rebuilt during rendering.
  document.addEventListener("click", (e) => {
    const target = e.target.closest("[data-action]");
    if (target) openDialog(target.dataset.action);
    const toggle = e.target.closest("[data-toggle-task]");
    if (toggle) {
      const task = state.tasks.find((t) => t.id === toggle.dataset.toggleTask);
      if (task) task.done = !task.done;
      render();
    }
    const del = e.target.closest("[data-delete-task]");
    if (del) {
      state.tasks = state.tasks.filter((t) => t.id !== del.dataset.deleteTask);
      render();
      toast("Task removed");
    }
    const gradeDel = e.target.closest("[data-delete-grade]");
    if (gradeDel) {
      state.grades = state.grades.filter(
        (g) => g.id !== gradeDel.dataset.deleteGrade,
      );
      state.gradesComputed = false;
      render();
      toast("Grade row removed");
    }
    const classDel = e.target.closest("[data-delete-class]");
    if (classDel) {
      state.classes = state.classes.filter(
        (c) => c.id !== classDel.dataset.deleteClass,
      );
      render();
      toast("Class removed");
    }
  });
  $$(".filter-tab").forEach((tab) =>
    tab.addEventListener("click", () => {
      $$(".filter-tab").forEach((t) =>
        t.classList.toggle("selected", t === tab),
      );
      renderTasks();
    }),
  );
  $("#expense-filter").addEventListener("change", renderExpenses);
  $("#grade-rows").addEventListener("change", (e) => {
    const field = e.target.closest("[data-grade-row]");
    if (!field) return;
    const row = state.grades.find((g) => g.id === field.dataset.gradeRow);
    if (row) {
      const property = field.dataset.gradeField;
      row[property] = property === "category"
        ? field.value
        : field.value === ""
          ? ""
          : Math.max(0, Number(field.value) || 0);
      if (property !== "category") {
        state.gradesComputed = false;
        $$(".grade-raw").forEach((cell) => (cell.textContent = "-"));
      }
      renderGradeSummary();
      persist();
    }
```

```
  });
  $("#add-grade-row").addEventListener("click", () => {
    state.grades.push({
      id: makeId(),
      category: GRADE_CATEGORIES[0],
      score: "",
      total: "",
    });
    state.gradesComputed = false;
    renderGrades();
    persist();
    $('#grade-rows tr:last-child [data-grade-field="score"]')?.focus();
  });
  $("#compute-grade").addEventListener("click", () => {
    state.gradesComputed = true;
    renderGrades();
    persist();
  });

  // 5. Shared entry dialog
  // Cancel and close only dismiss the dialog; the form submit handler is reserved for Save.
  const dialog = $("#entry-dialog"),
    form = $("#entry-form");
  document.addEventListener("click", (e) => {
    if (e.target.closest("[data-close-dialog]")) dialog.close();
  });
  function field(
    label,
    name,
    type = "text",
    value = "",
    options = null,
    extra = "",
  ) {
    if (options)
      return `<div class="dialog-field"><label for="field-${name}">${label}</label><select id="field-${name}" name="${name}" ${extra}>${options.map((o) => `<option value="${escapeHtml(o)}" ${o === value ? "selected" : ""}>${escapeHtml(o)}</option>`).join("")}</select></div>`;
    return `<div class="dialog-field"><label for="field-${name}">${label}</label><input id="field-${name}" name="${name}" type="${type}" value="${escapeHtml(value)}" ${type === "number" ? 'min="0" step="1"' : ""} ${extra} required></div>`;
  }
  // One dialog builder supplies the fields and save action for each add/edit interaction.
  function openDialog(action) {
    const config = {
      "add-task": {
        title: "Add a task",
        subtitle: "A SMALL THING, DONE WELL",
        fields: `${field("What needs doing?", "title", "text", "", "", 'placeholder="e.g. Review lecture notes"')}<div class="dialog-grid">${field("Due", "due", "text", "Today · Anytime", "", 'placeholder="Today · 3:00 PM"')}${field("Priority", "priority", "text", "Medium", ["Low", "Medium", "High"])}</div>`,
        save: (data) =>
          state.tasks.unshift({
            id: makeId(),
            title: data.get("title").trim(),
            due: data.get("due").trim() || "Today",
            priority: data.get("priority"),
            done: false,
          }),
      },
      "add-expense": {
        title: "Add an expense",
        subtitle: "MONEY CHECK-IN",
        fields: `${field("What did you spend on?", "name", "text", "", "", 'placeholder="e.g. Lunch"')}<div class="dialog-grid">${field("Amount (₱)", "amount", "number", "", "", 'min="1"')}${field("Category", "category", "text", "Food", ["Food", "Transport", "School", "Personal", "Other"])}</div>${field("Date", "date", "date", today.toISOString().slice(0, 10))}`,
        save: (data) =>
          state.expenses.unshift({
            id: makeId(),
            name: data.get("name").trim(),
            amount: Number(data.get("amount")),
            category: data.get("category"),
            date: data.get("date"),
          }),
      },
      "add-class": {
        title: "Add a class",
        subtitle: "BUILD YOUR WEEK",
        fields: `${field("Subject", "name", "text", "", "", 'placeholder="e.g. Chemistry"')}<div class="dialog-grid">${field("Day", "day", "text", "Mon", DAYS)}${field("Start time", "time", "text", "9:00", ["8:00", "9:00", "10:00", "11:00", "12:00", "1:00", "2:00", "3:00", "4:00"])}</div>${field("Room or location", "room", "text", "", "", 'placeholder="e.g. Science Hall · 204"')}`,
        save: (data) =>
          state.classes.push({
            id: makeId(),
            day: data.get("day"),
            time: data.get("time"),
            name: data.get("name").trim(),
            room: data.get("room").trim() || "Room not set",
          }),
      },
      "edit-budget": {
        title: "Monthly allowance",
        subtitle: "SET YOUR MONTHLY PLAN",
        fields: field(
          "Allowance (₱)",
          "allowance",
          "number",
          state.allowance,
          "",
          'min="0"',
        ),
        save: (data) => (state.allowance = Number(data.get("allowance"))),
      },
    }[action];
    if (!config) return;
    $("#dialog-title").textContent = config.title;
    $("#dialog-eyebrow").textContent = config.subtitle;
    $("#dialog-fields").innerHTML = config.fields;
    form.onsubmit = (e) => {
      e.preventDefault();
      if (!form.reportValidity()) return;
```

```
      const data = new FormData(form);
      config.save(data);
      render();
      dialog.close();
      toast(
        action === "edit-budget"
          ? "Allowance updated"
          : "Added to your planner",
      );
    };
    dialog.showModal();
    setTimeout(() => $("#dialog-fields input")?.focus(), 30);
  }

  // 6. Reset and first page load
  $("#reset-data").addEventListener("click", () => {
    if (
      confirm(
        "Restore the original sample planner? Your current changes will be replaced.",
      )
    ) {
      state = structuredClone(seed);
      render();
      navigate("overview");
      toast("Sample planner restored");
    }
  });
  const initial = location.hash.replace("#", "");
  navigate(
    ["overview", "planner", "classes", "budget", "grades"].includes(initial)
      ? initial
      : "overview",
  );
  render();
})();
```